

HW9 Results Report

This report contains the required test evidence, result summaries, graphs, and discussion for the HW9 distributed databases submission.

How the Code Works

System structure

The project has two separate database implementations:

- `leader-follower/main.go`
- `leaderless/main.go`

Both expose the same basic client API:

- `POST /set`
- `GET /get`

Both keep all data in memory only, as required by the assignment. Each stored key carries both a value and a logical version number.

Leader-Follower: what happens on writes

There are five total nodes: one leader and four followers.

When a write arrives at the leader:

1. The leader validates the request and rejects empty keys.
2. The leader assigns the next logical version for that key.
3. The leader stores the new value locally.
4. The leader sends replication messages to followers sequentially, sleeping 200ms after each message. Each follower sleeps 100ms before acknowledging.
5. The leader waits for enough acknowledgements to satisfy the configured `W`.

The three write cases are implemented like this:

- `W=5` The leader waits for acknowledgements from all four followers before replying `201 Created`. If any follower fails to ack, the write returns an error (503) instead of falsely acknowledging.
- `W=1` The leader counts its own local write as the full write quorum, returns immediately, and continues follower replication asynchronously.
- `W=3` The leader waits until the write exists on the leader and any two followers ($\text{leader} + 2 = 3$), then it returns and finishes replicating to the remaining followers in the background. If fewer than 2 followers ack, the write fails with 503.

The tricky part here is making sure the service does not falsely acknowledge a write that did not actually reach the required quorum. The code handles this by returning an error instead of success if the required acknowledgements are not received.

Leader-Follower: what happens on reads

Reads can arrive at any node. When a read arrives at a follower, the follower forwards it to the leader, so that the leader's quorum-read logic always applies regardless of which node the client contacted.

When the leader processes a read:

- **R=1** The leader returns its local value immediately.
- **R=3** The leader queries two followers in parallel (each sleeps 50ms), compares their versions with its local version, and returns the highest version it sees.
- **R=5** The leader queries all four followers in parallel and returns the newest version among all replicas.

The tricky part is freshness: a follower may be behind during propagation, so quorum reads must compare versions and choose the newest response instead of trusting the first response that arrives.

Leaderless: what happens on writes

In the leaderless system, there is no distinguished leader. Any node can receive a write.

When a write arrives:

1. That node becomes the write coordinator for that request.
2. It validates the request.
3. It assigns a logical version (using a node-rank-encoded counter to avoid collisions across concurrent coordinators) and stores the value locally.
4. It sends the write to every other node sequentially, sleeping 200ms after each message. Each peer sleeps 100ms before acknowledging.
5. Because this implementation uses **W=N**, it only replies **201 Created** after every other node acknowledges. If any peer fails, the write returns 503.

The tricky part here is version ordering. Since multiple nodes can coordinate writes independently, version numbers must still be comparable across the cluster. The implementation encodes a node-specific rank component into the version so ties are avoided and "most recent" comparisons stay stable.

Leaderless: what happens on reads

For the leaderless case, **R=1**, so a node simply returns its own local state.

That is exactly why stale reads happen in this design: during propagation, another node may not yet have the newest write, but it is still allowed to answer the read.

Delays, inconsistency windows, and testing hooks

To make inconsistency windows visible, the code deliberately adds the assignment's required delays:

- The sender sleeps **200ms** after each replication message
- The receiving replica sleeps **100ms** before acknowledging a replicated write
- Follower quorum-read responses sleep **50ms**

Without those delays, the race windows would be too small to observe clearly in testing.

There is also a `local_read` endpoint used only for testing. That endpoint returns the node's local state directly, without quorum coordination, which makes it possible to prove that temporary inconsistency exists inside the system while writes are still propagating.

Error handling and maintenance notes

If I were handing this code to a friend to maintain, the main things I would point out are:

- The code assumes in-memory storage only. Restarting a node loses all data.
- Logical versions are central to correctness. Any future change to replication or read coordination must preserve monotonic version comparison.
- The hardest bugs in this code are not syntax bugs, but timing bugs: false acknowledgements, stale reads, and ordering mistakes under concurrency.
- `local_read` is intentionally not a normal client API. It is there to expose internal inconsistency during tests.
- The most important failure rule is that a write should not return success unless the required quorum for that configuration was actually satisfied.

How the Load Tester Works

The load tester (`loadtester/main.go`) is a concurrent Go program that generates read and write traffic at configurable ratios.

Guaranteeing temporal locality

The assignment requires that reads and writes to the same key cluster closely together in time. The load tester achieves this by using a **small key space**: by default only 10 distinct keys (`key_0` through `key_9`). With 10 concurrent workers all randomly choosing from 10 keys, the probability that a read hits a recently-written key is very high — roughly 1 in 10 per operation. This means at any moment, multiple workers are likely reading and writing overlapping keys within milliseconds of each other.

This is the simplest effective strategy. It does not require complex scheduling or time-windowed algorithms. The small key pool naturally forces temporal clustering because every worker is drawing from the same small set.

How each request works

Each worker loops, randomly deciding read vs. write based on the configured write percentage (1%, 10%, 50%, or 90%). For writes, it picks a random key from the pool, generates a unique value, and sends `POST /set` to a write node. For reads, it picks a random key and sends `GET /get` to a random read node.

Staleness detection

The client maintains a `VersionTracker` that records the latest version written for each key. When a read response arrives, the client compares the returned version against the latest known version for that key. If the read version is lower, the read is marked as stale. This is how the stale-read counts in the results table are computed.

Metrics collected

For each request the tester records: operation type (read/write), key, latency in milliseconds, HTTP status code, version number, stale flag, timestamp, and the time interval between this read and the most recent write to the same key. All of this is written to a CSV file for graphing.

Tests Performed

The required consistency and inconsistency behaviors were tested for both database types. The tests are self-contained: `TestMain` builds the Go binaries, starts local clusters as subprocesses, runs all tests, and cleans up afterward. No Docker is needed to run the tests.

Leader-Follower tests

- Read from the leader after an acknowledged write: confirms consistent data
- Read from a follower after an acknowledged write (W=5): confirms consistent data
- Show inconsistency through `local_read` during the W=1 propagation window
- Verify quorum read (R=3) returns the latest value
- Verify quorum write (W=3) acks at least three nodes

Leaderless tests

- Show inconsistency while a write is still propagating to peers
- Read from the write coordinator after acknowledgement: confirms consistent data
- Read from all nodes after coordinator acknowledgement (W=N): confirms all consistent

Test Execution Screenshot

Go Test Output

```
2026/04/06 01:00:48 Starting FOLLOWER node follower1 on :8081 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADER node leader on :8080 (W=5, R=1, followers=[localhost:8081 localhost:8082 localhost:8083 localhost:8084])
2026/04/06 01:00:48 Starting FOLLOWER node follower2 on :8082 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower3 on :8083 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower4 on :8084 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADERLESS node node2 on :9081 (peers=[localhost:9080 localhost:9082 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node1 on :9080 (peers=[localhost:9081 localhost:9082 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node3 on :9082 (peers=[localhost:9080 localhost:9081 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node4 on :9083 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node5 on :9084 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9083])
=== RUN TestLeaderReadFromLeaderConsistent
2026/04/06 01:00:49 Configuration updated: W=5, R=1
--- PASS: TestLeaderReadFromLeaderConsistent (1.33s)
=== RUN TestLeaderFollowerGetConsistentAfterAck
2026/04/06 01:00:50 Configuration updated: W=5, R=1
--- PASS: TestLeaderFollowerGetConsistentAfterAck (1.37s)
=== RUN TestLeaderLocalReadShowsInconsistencyWindow
2026/04/06 01:00:51 Configuration updated: W=1, R=1
--- PASS: TestLeaderLocalReadShowsInconsistencyWindow (0.10s)
=== RUN TestLeaderQuorumReadReturnsLatestValue
2026/04/06 01:00:51 Configuration updated: W=3, R=3
--- PASS: TestLeaderQuorumReadReturnsLatestValue (0.76s)
=== RUN TestLeaderQuorumWriteAcksAtLeastThreeNodes
2026/04/06 01:00:52 Configuration updated: W=3, R=3
--- PASS: TestLeaderQuorumWriteAcksAtLeastThreeNodes (0.71s)
=== RUN TestLeaderlessInconsistencyWindow
--- PASS: TestLeaderlessInconsistencyWindow (1.23s)
=== RUN TestLeaderlessCoordinatorConsistentAfterAck
--- PASS: TestLeaderlessCoordinatorConsistentAfterAck (1.23s)
=== RUN TestLeaderlessAllNodesConsistentAfterAck
--- PASS: TestLeaderlessAllNodesConsistentAfterAck (1.28s)
PASS
ok  hw9/tests 12.475s
```

All 8 tests pass (PASS ok hw9/tests 12.475s).

Result Summary

Configuration	Write %	Read %	Stale Reads	Stale %	Write Median (ms)	Read Median (ms)	Throughput
Leader-Follower W=5, R=1	1	99	0	0.0	1216.81	6.11	311.7 req/s
Leader-Follower W=5, R=1	10	90	6	0.7	1211.47	1.05	90.6 req/s
Leader-Follower W=5, R=1	50	50	75	15.3	1211.40	1.14	16.2 req/s
Leader-Follower W=5, R=1	90	10	26	26.0	1213.21	1.36	9.2 req/s
Leader-Follower W=1, R=5	1	99	0	0.0	1211.86	1.10	392.0 req/s
Leader-Follower W=1, R=5	10	90	31	3.4	1211.08	1.05	81.6 req/s
Leader-Follower W=1, R=5	50	50	68	13.6	1211.93	1.13	16.5 req/s
Leader-Follower W=1, R=5	90	10	17	16.7	1212.67	1.41	9.2 req/s
Leader-Follower W=3, R=3	1	99	1	0.1	1212.56	1.53	670.4 req/s
Leader-Follower W=3, R=3	10	90	3	0.3	1210.20	0.96	81.5 req/s
Leader-Follower W=3, R=3	50	50	74	14.1	1211.82	1.14	17.2 req/s
Leader-Follower W=3, R=3	90	10	16	14.8	1212.43	1.40	9.2 req/s
Leaderless W=N, R=1	1	99	8	0.8	1217.86	6.01	495.0 req/s
Leaderless W=N, R=1	10	90	278	30.9	1210.77	0.64	74.6 req/s
Leaderless W=N, R=1	50	50	220	45.5	1210.91	1.01	15.9 req/s
Leaderless W=N, R=1	90	10	53	55.2	1211.65	1.15	9.1 req/s

Discussion of Results

Which leader-follower type does best with each read/write ratio?

Using throughput as the main comparison among the three leader-follower configurations:

- **1% writes / 99% reads:** $W=3$, $R=3$ performed best (670.4 req/s). It benefits from moderate write cost and moderate read cost. Since writes are rare, the quorum overhead barely matters, and the balanced approach lets reads flow smoothly.
- **10% writes / 90% reads:** $W=5$, $R=1$ performed best (90.6 req/s). With $R=1$, reads are extremely cheap (local to the leader), and the relatively low write volume means the expensive $W=5$ writes don't dominate.
- **50% writes / 50% reads:** $W=3$, $R=3$ performed best (17.2 req/s). The balanced quorum avoids the extreme cost of full replication on every write ($W=5$) while still maintaining reasonable read freshness.
- **90% writes / 10% reads:** $W=1$, $R=5$ and $W=3$, $R=3$ were nearly tied (~ 9.2 req/s). When writes dominate, the bottleneck is write latency, and since injected delays make all writes cost roughly the same $\sim 1.2s$ regardless of W , throughput converges.

Why those results happened

The graphs are not just showing random variation; the behavior follows directly from the replication rules.

$W=5$, $R=1$ makes reads cheap but writes expensive. This is why it does well when writes are rare: the system benefits from very simple reads, and the cost of waiting for every follower is not paid often. As write percentage increases, this configuration becomes less attractive because every write must fully replicate before returning. The latency histograms for $W=5, R=1$ at 90% writes show the write distribution tightly clustered around 1.2 seconds, confirming the sequential replication delay dominates.

$W=1$, $R=5$ shifts the cost in the other direction. Writes can return immediately because the leader only needs its own local write to satisfy $W=1$, but reads need broader coordination to be logically fresh. Under heavy write pressure, this helps keep throughput competitive. The stale-read percentage for $W=1, R=5$ is notably lower than leaderless because the quorum-read logic on the leader still fetches and compares versions from followers, catching most staleness. However, at 10% writes it already shows 3.4% stale reads — the async replication gap is real.

$W=3$, $R=3$ is the most balanced design. It avoids the extreme cost of waiting for all replicas on every write, and it also avoids relying on a single local copy for every read. That is why it performs especially well in the mixed workloads: it trades some read and write simplicity for better overall balance. The quorum overlap ($W + R = 6 > N = 5$) guarantees that at least one node involved in a quorum read was also part of the quorum write, which is why stale-read rates stay low.

Leaderless $W=N$, $R=1$ shows the highest stale-read percentages by a large margin (30.9% at 10% writes, 55.2% at 90% writes). That happens because reads go directly to whichever node the client contacts, and those nodes do not coordinate before responding. During write propagation (which takes $\sim 1.2s$ across 4 peers), a read can easily hit a lagging replica. This is exactly the inconsistency window the assignment expects to see. The `rw_interval` histograms for leaderless confirm that reads often arrive within milliseconds of a write to the same key, well within the propagation window.

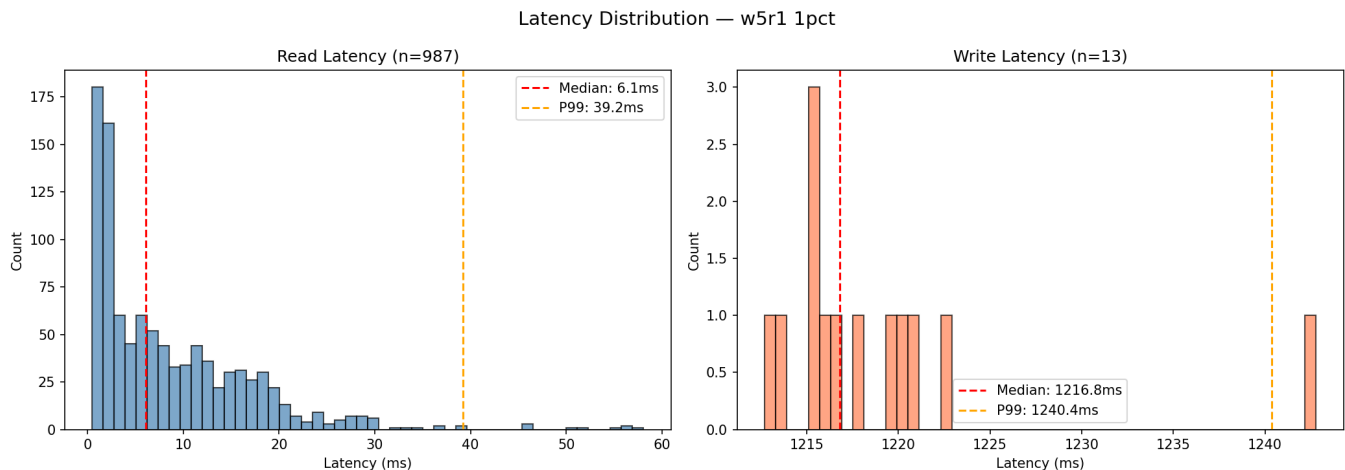
Another important point is that the injected delays dominate write latency across all configurations. That is why the median write times are all clustered near the same 1.2 second range regardless of W value. The more meaningful differences between configurations show up in stale-read rates, read latency distributions, and throughput under changing read/write mixes.

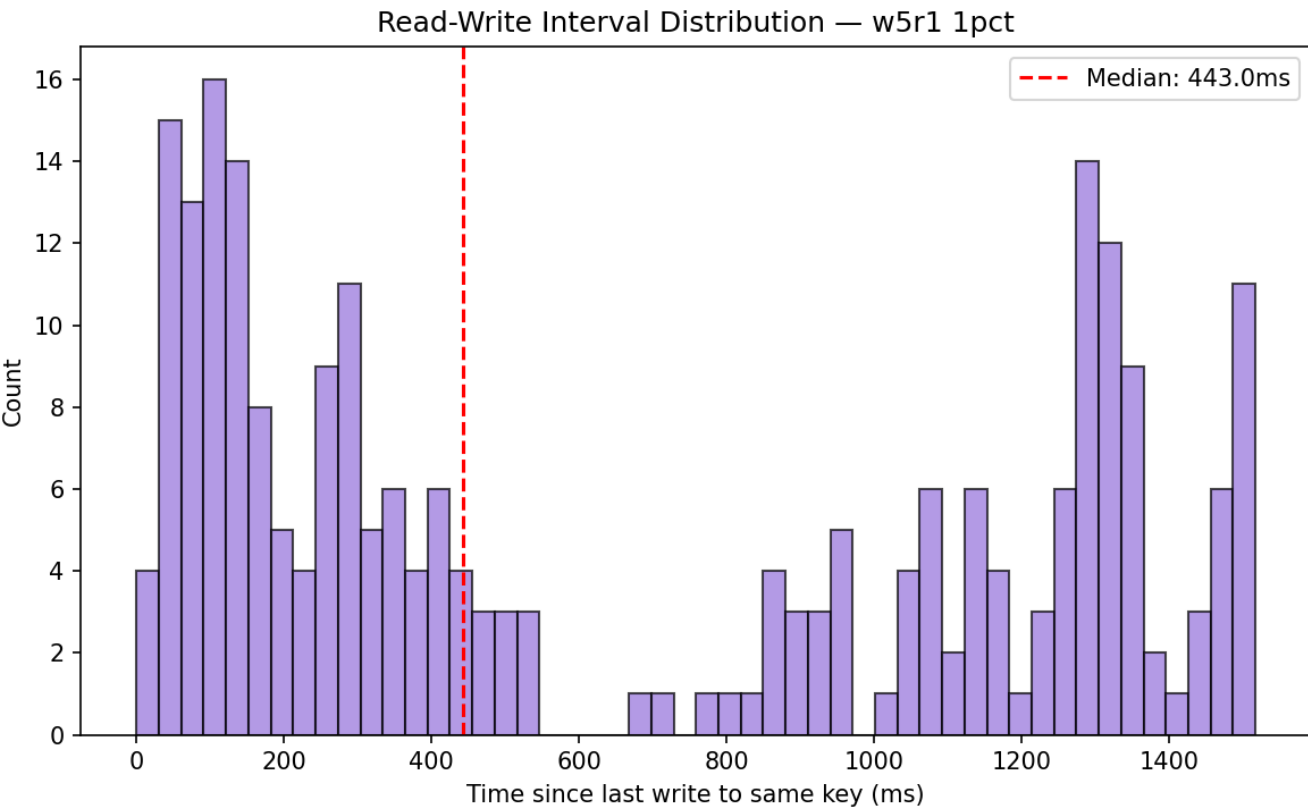
Which database is best for which kind of application?

- **Leader-Follower W=5, R=1:** Best for applications where reads should be simple and strongly up to date after acknowledged writes, and where write volume is relatively low. Examples: configuration stores, DNS-like lookup services, or caching layers where writes are infrequent but reads must always see the latest value.
- **Leader-Follower W=1, R=5:** Better when write responsiveness matters more and the system is willing to pay extra read coordination cost to recover freshness. Examples: write-heavy analytics ingestion or logging pipelines where data flows in fast but queries are less frequent and can tolerate slightly more latency.
- **Leader-Follower W=3, R=3:** Best general-purpose compromise for mixed workloads. The quorum overlap guarantees freshness while neither reads nor writes are prohibitively expensive. Examples: user-facing web applications, session stores, or general-purpose key-value caches with moderate read/write balance.
- **Leaderless W=N, R=1:** Best for applications that can tolerate temporary inconsistency and care more about flexible coordination or decentralized write handling. Examples: activity feeds, soft-state dashboards, approximate monitoring views, or distributed counters where brief staleness is acceptable.

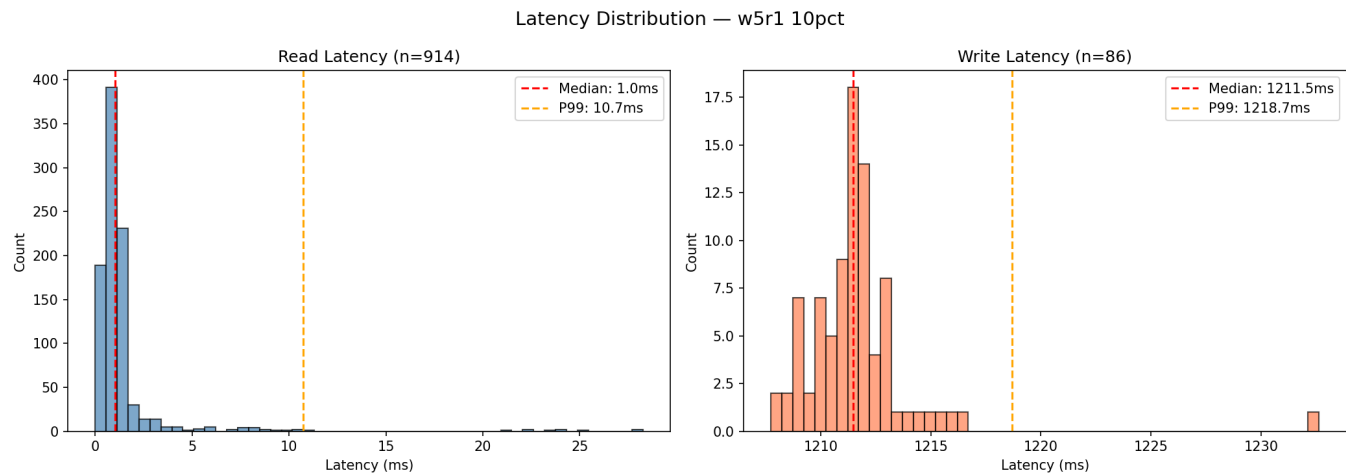
Graphs — Leader-Follower W=5, R=1

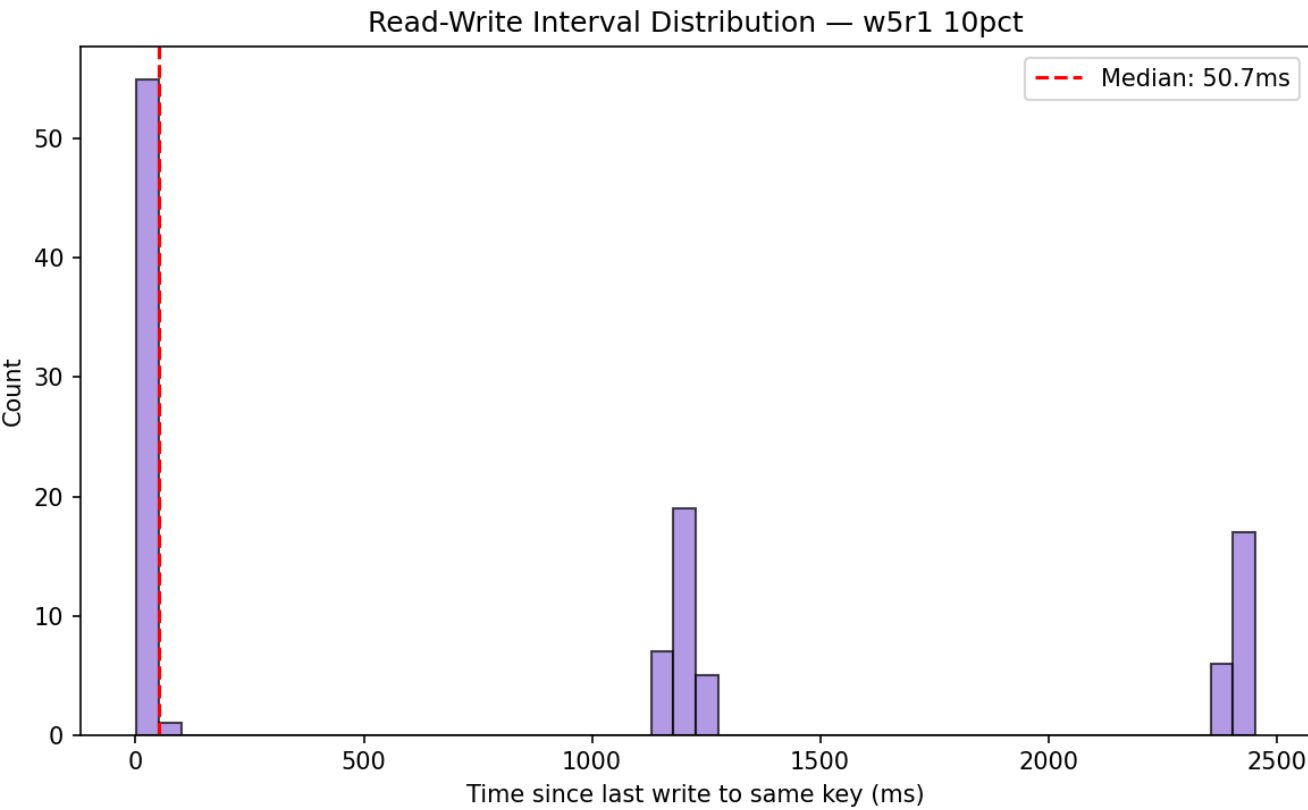
1% Writes / 99% Reads



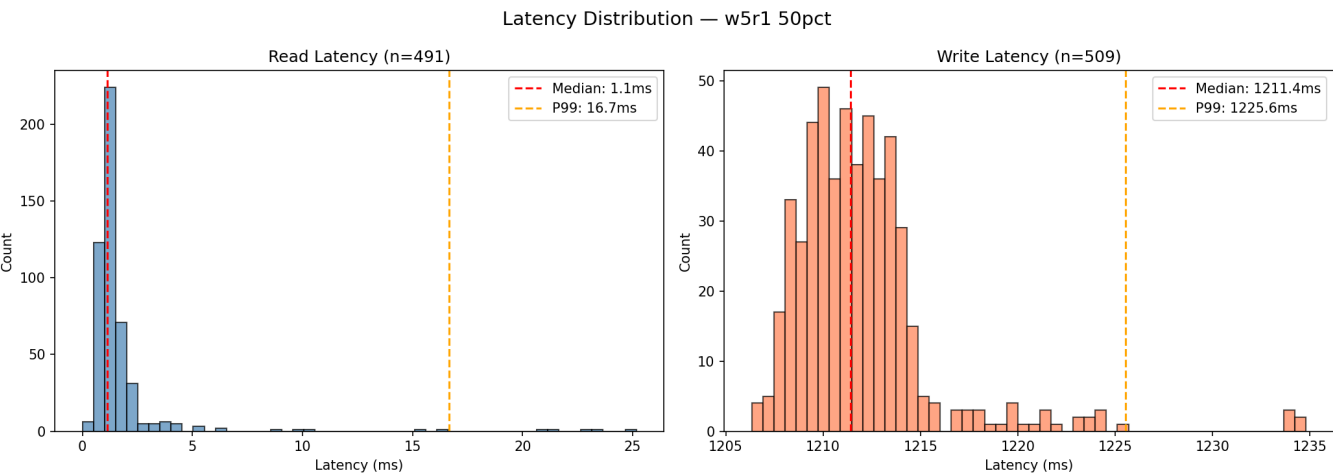


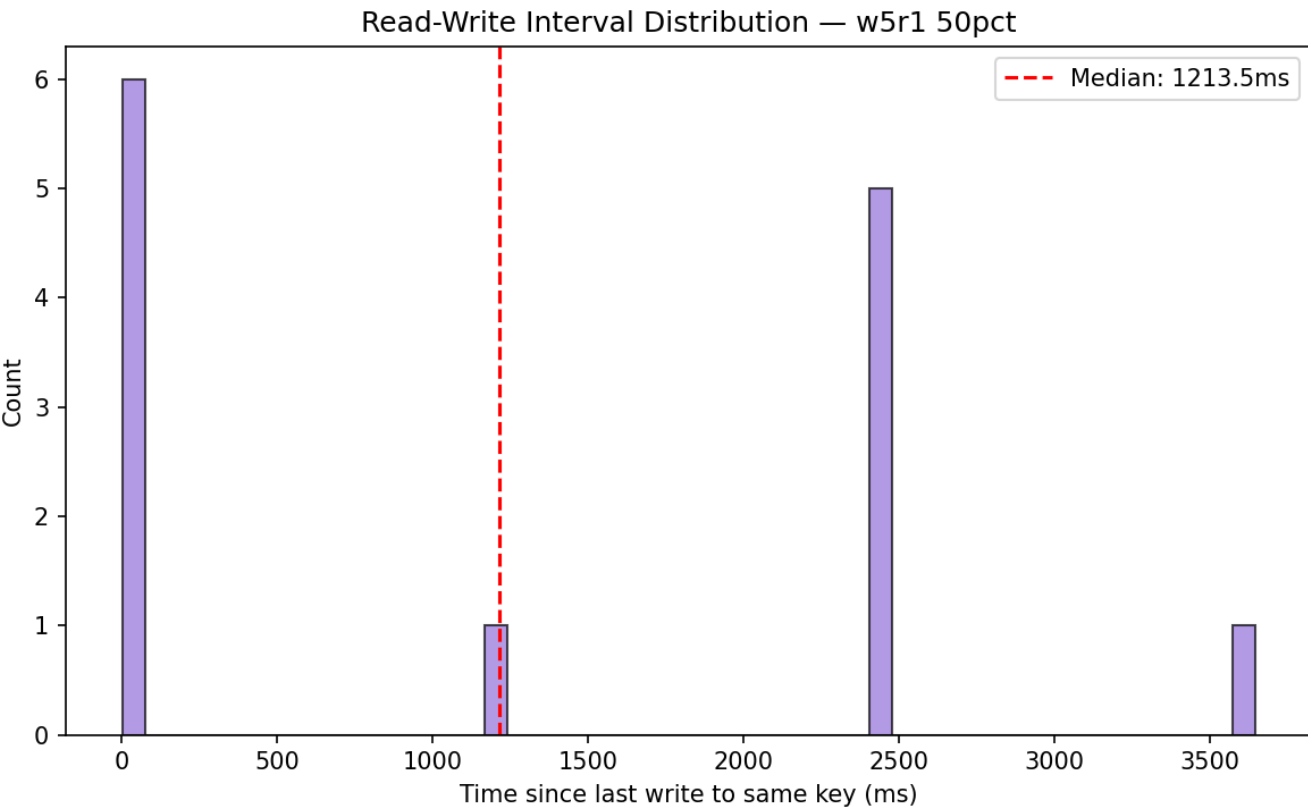
10% Writes / 90% Reads



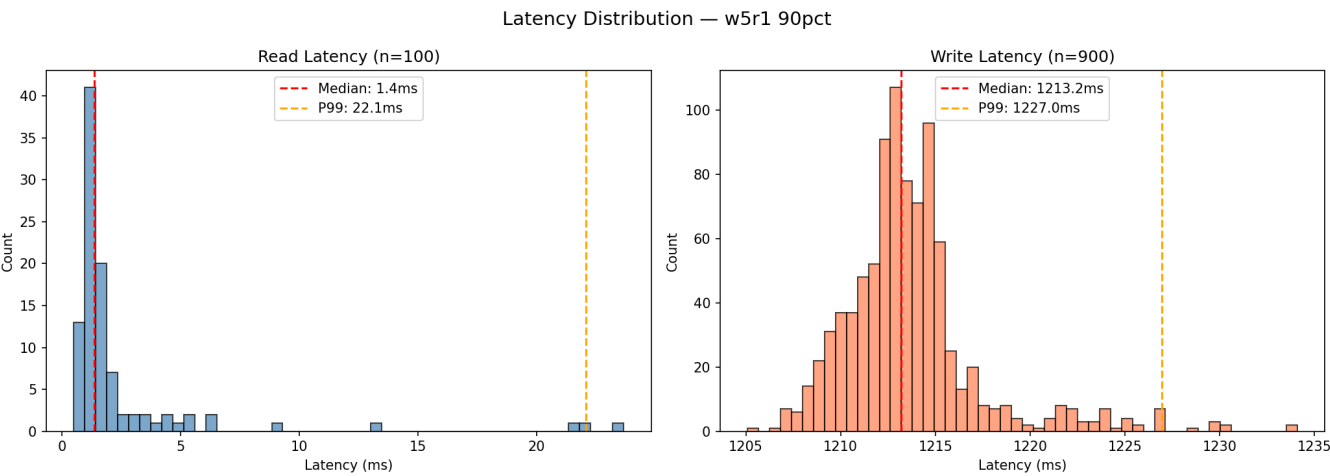


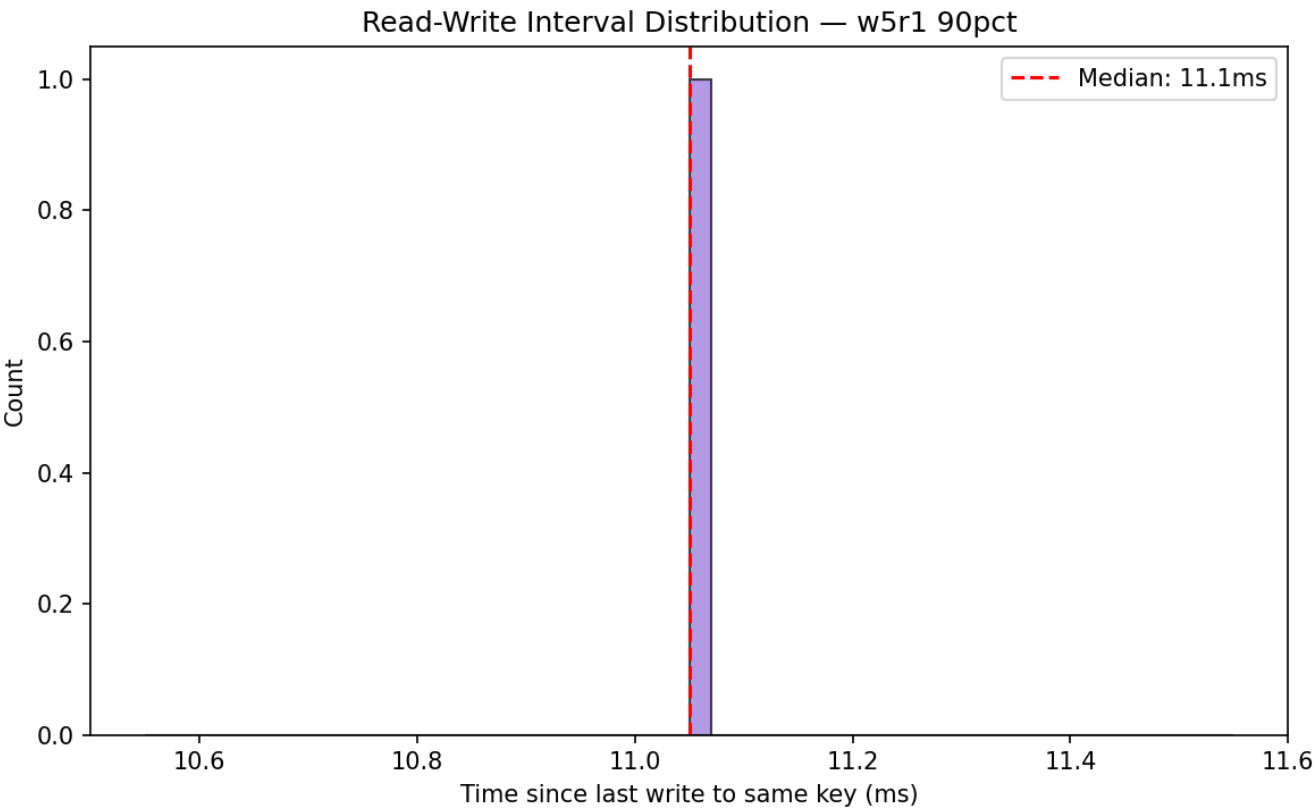
50% Writes / 50% Reads





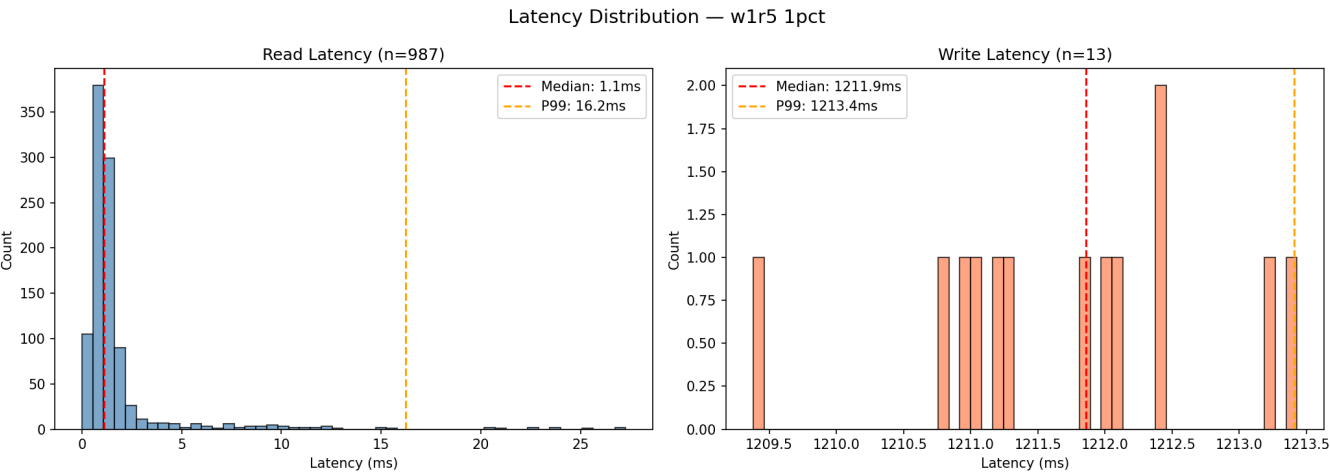
90% Writes / 10% Reads

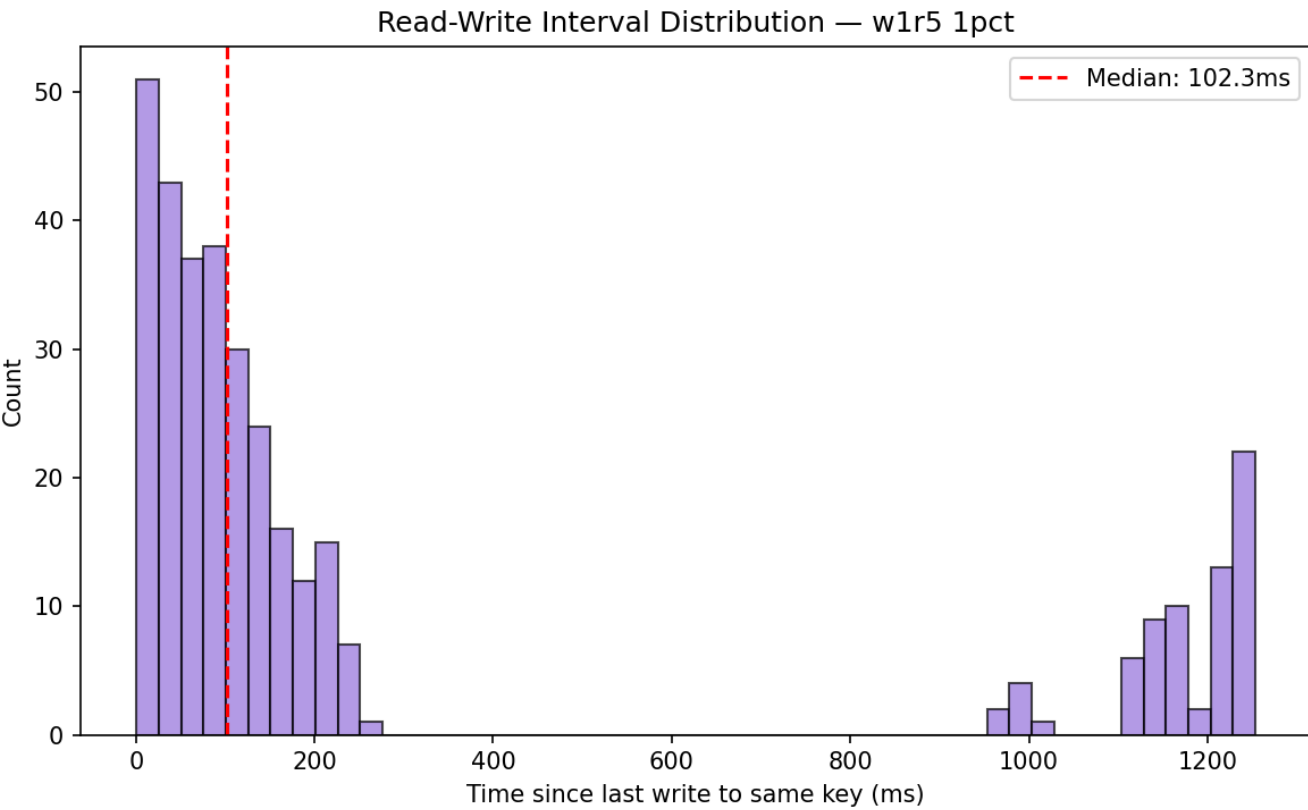




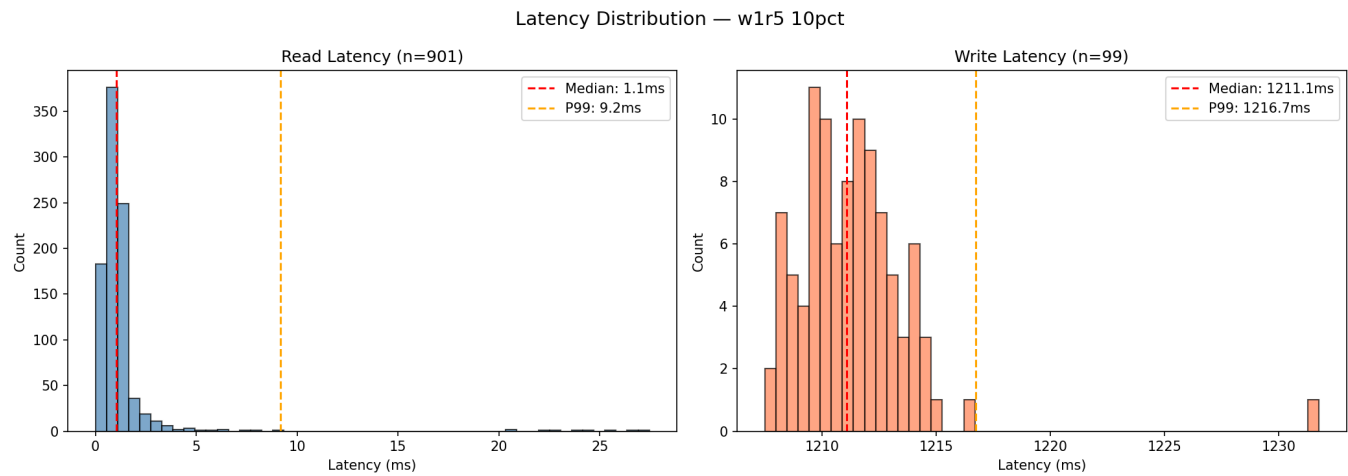
Graphs — Leader-Follower W=1, R=5

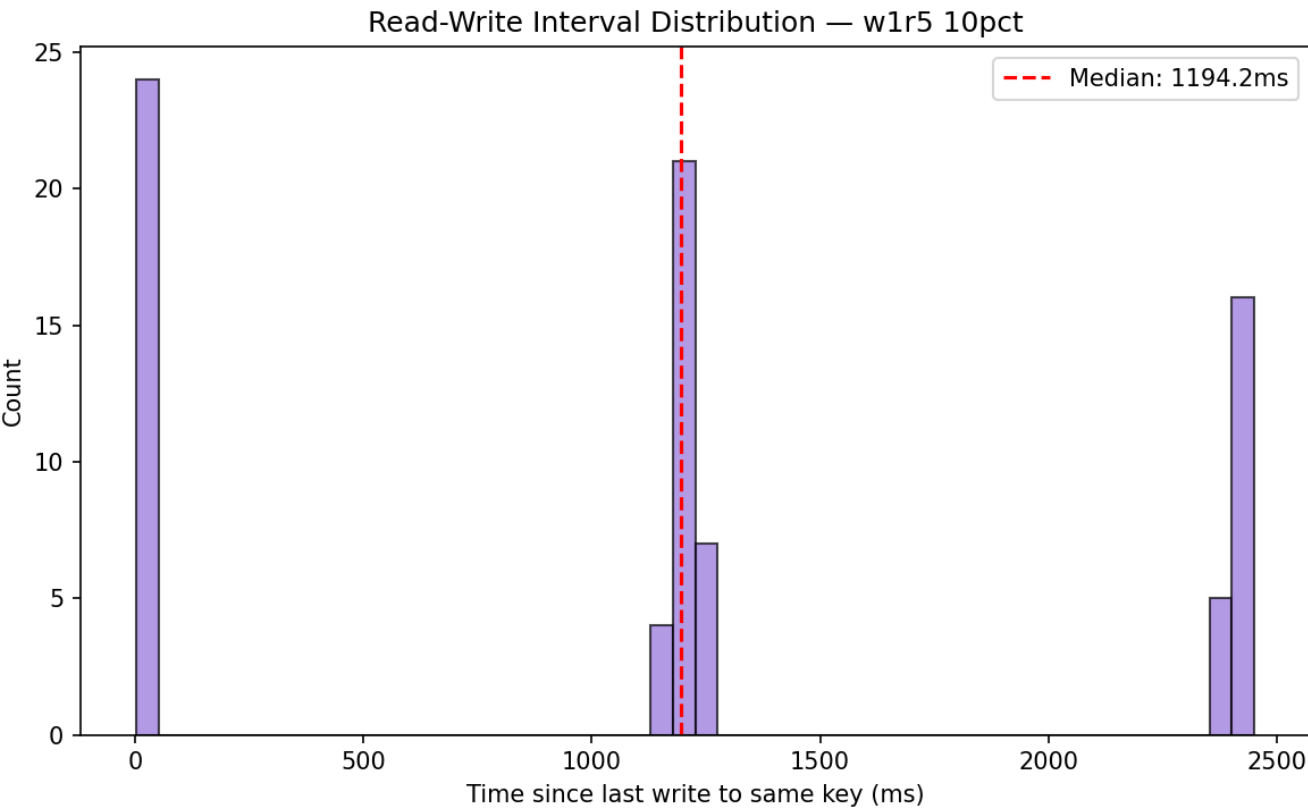
1% Writes / 99% Reads



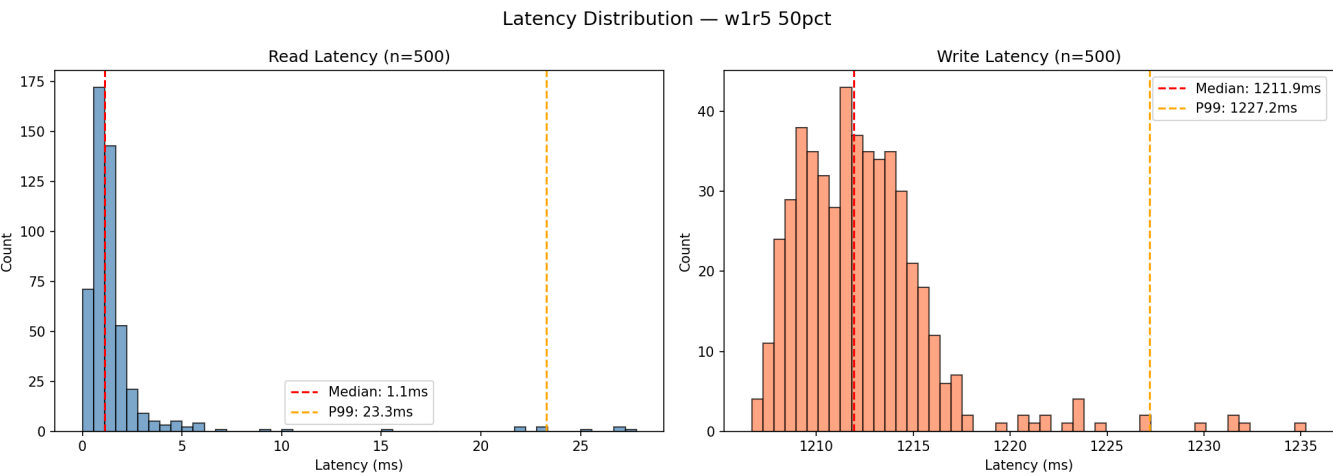


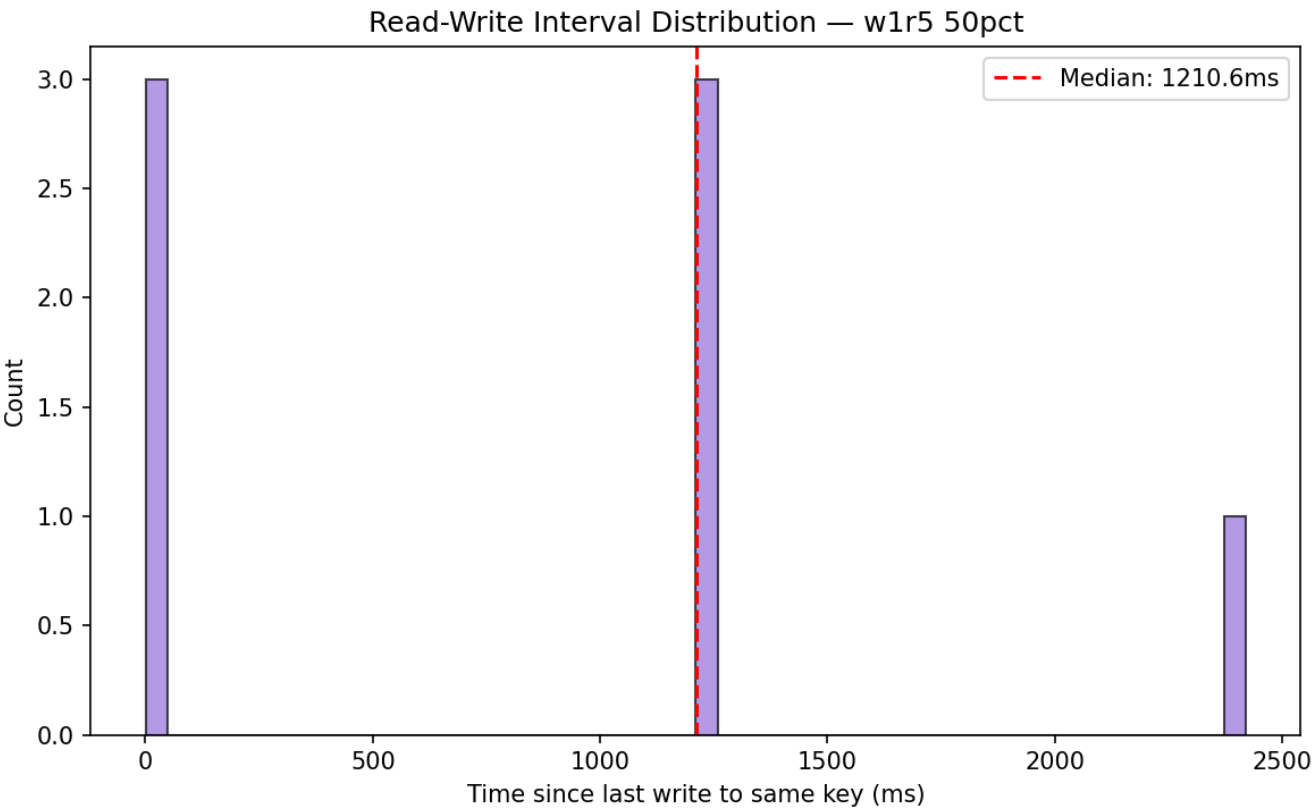
10% Writes / 90% Reads



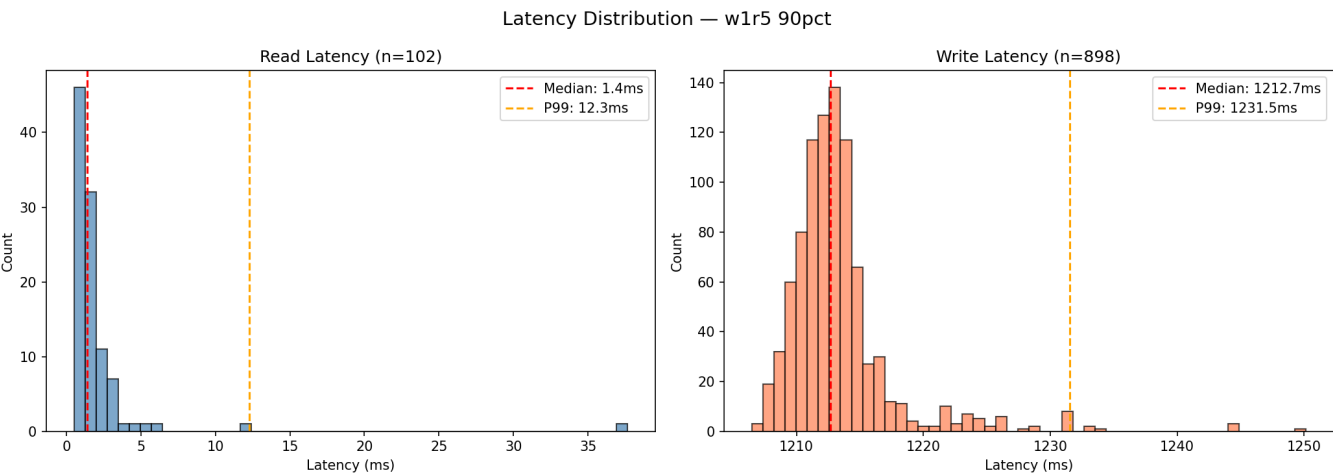


50% Writes / 50% Reads





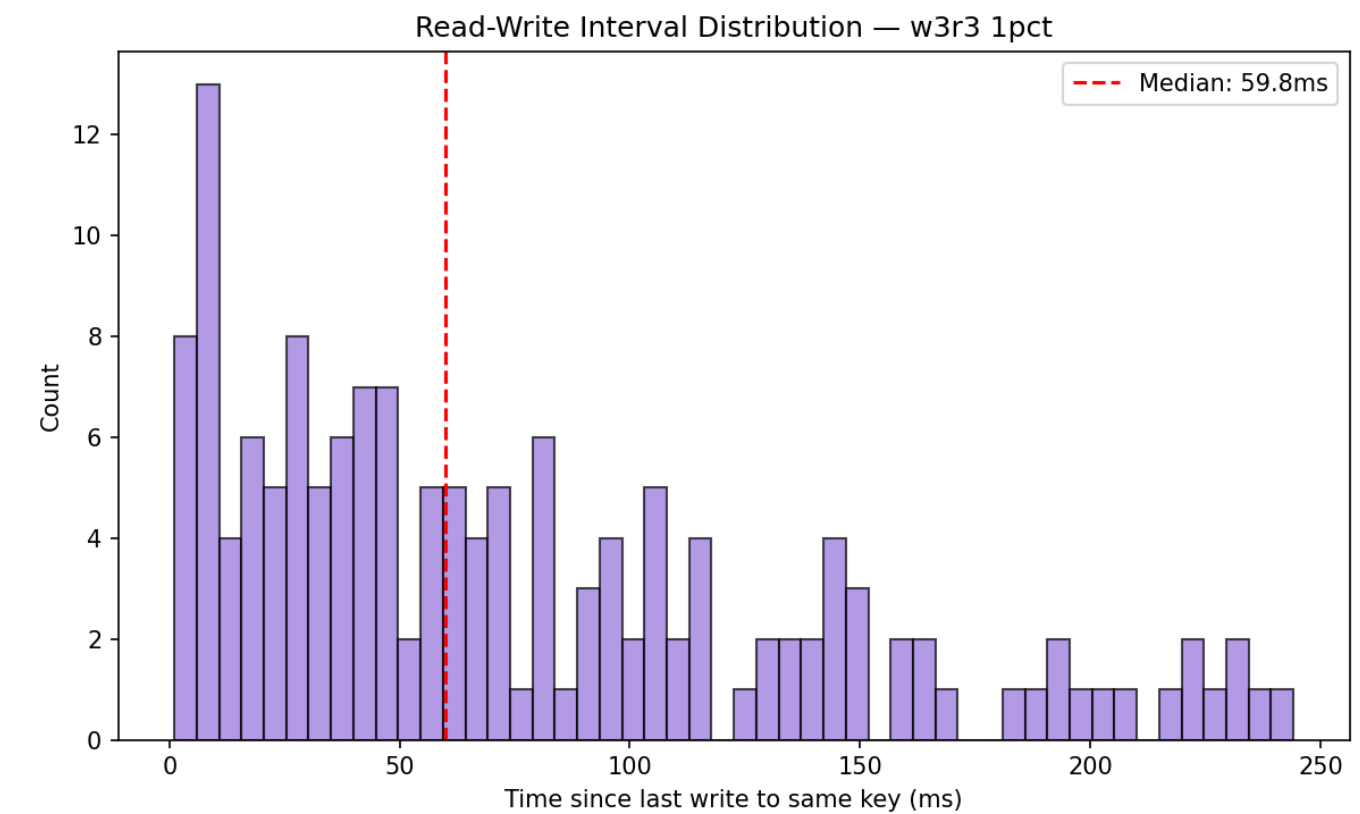
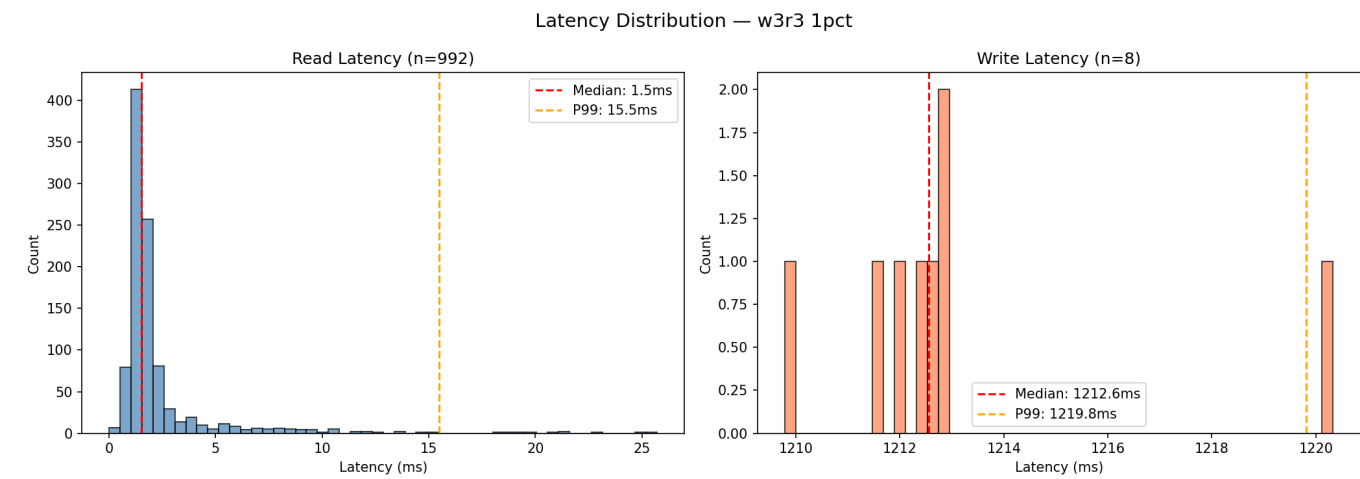
90% Writes / 10% Reads



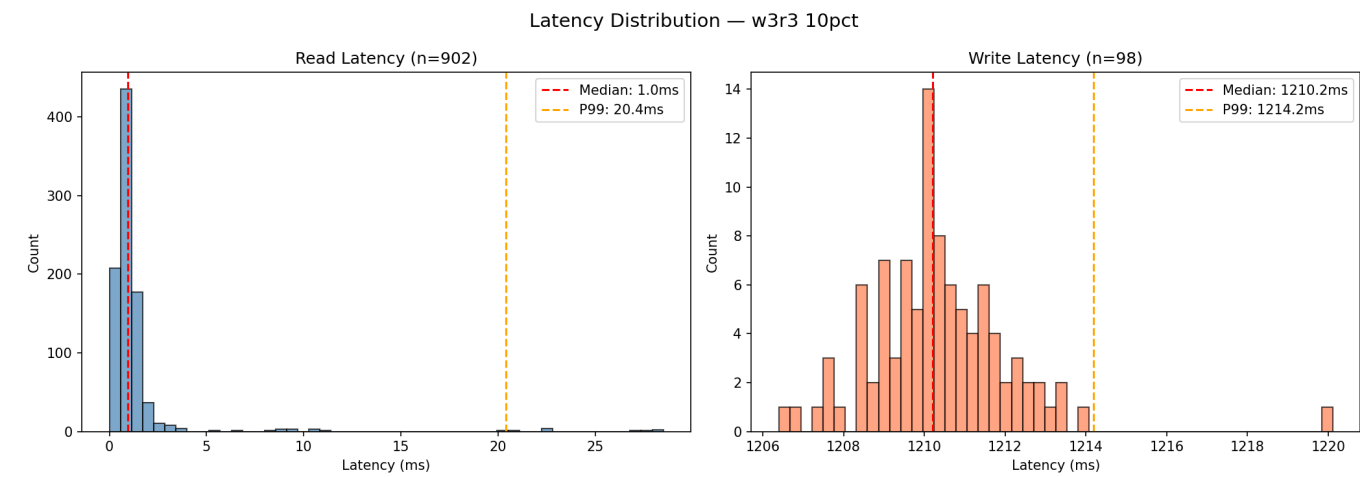
The read-write interval graph is not available for this ratio. With 90% writes and only 10% reads, the small number of reads that had a prior write to the same key was insufficient to produce a meaningful interval distribution.

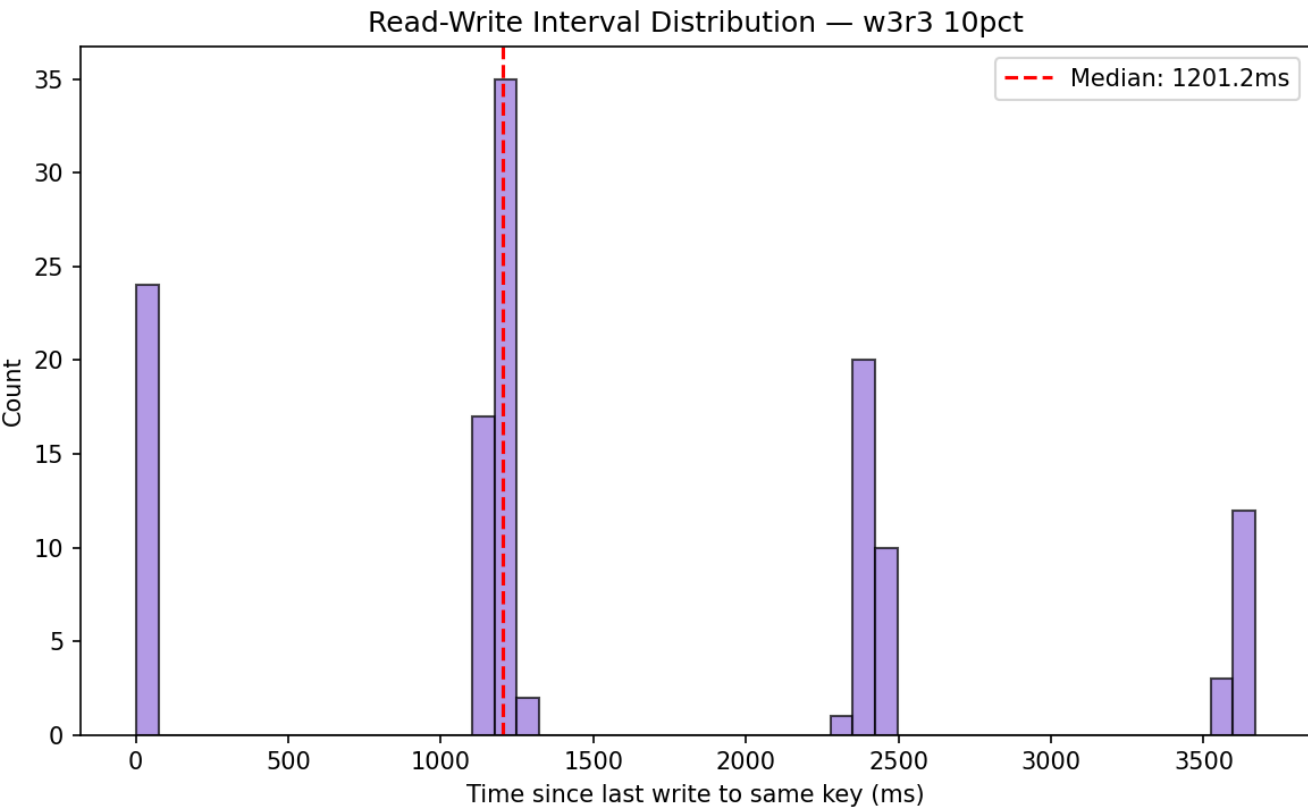
Graphs — Leader-Follower W=3, R=3 (Quorum)

1% Writes / 99% Reads

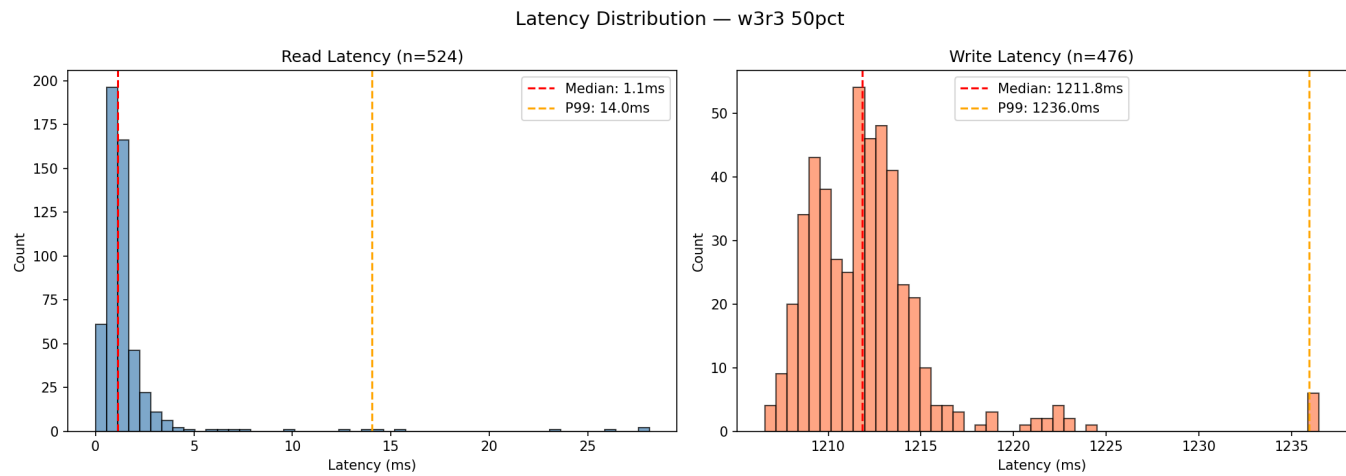


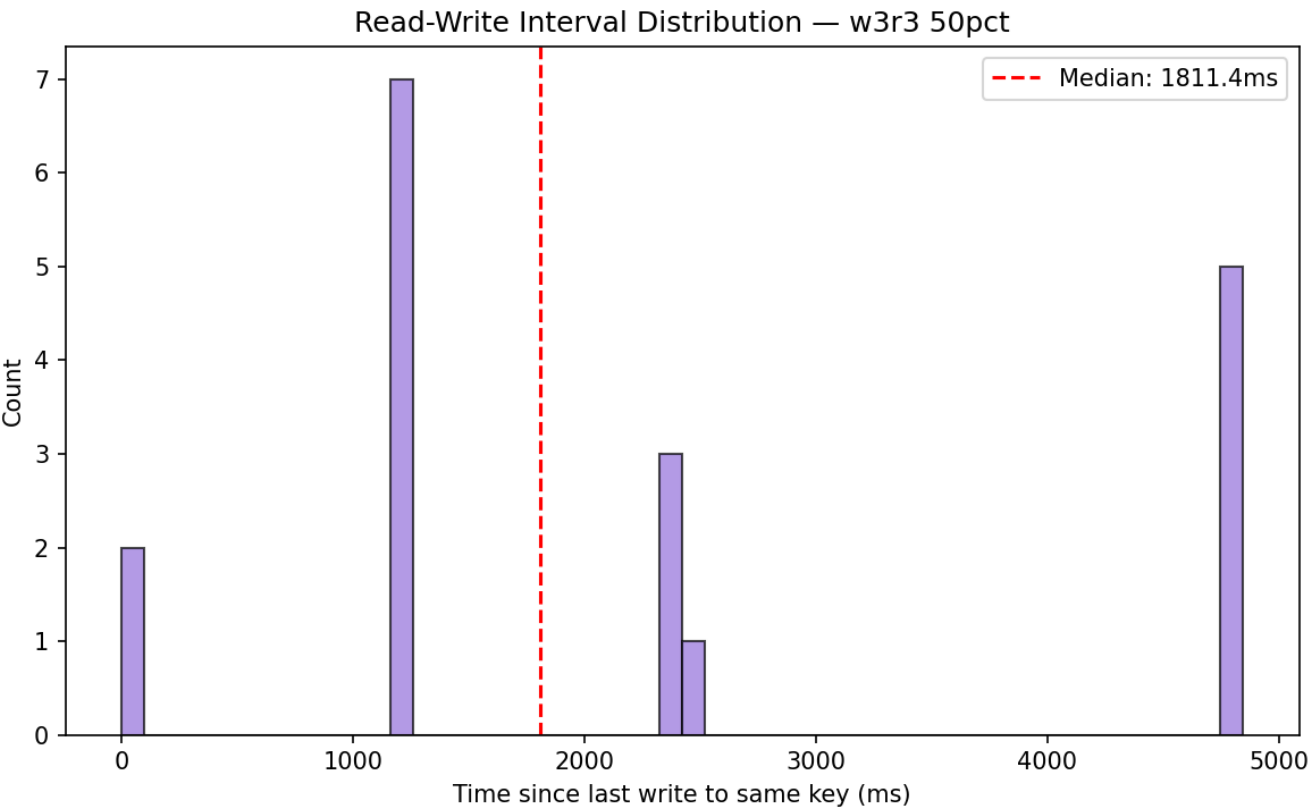
10% Writes / 90% Reads



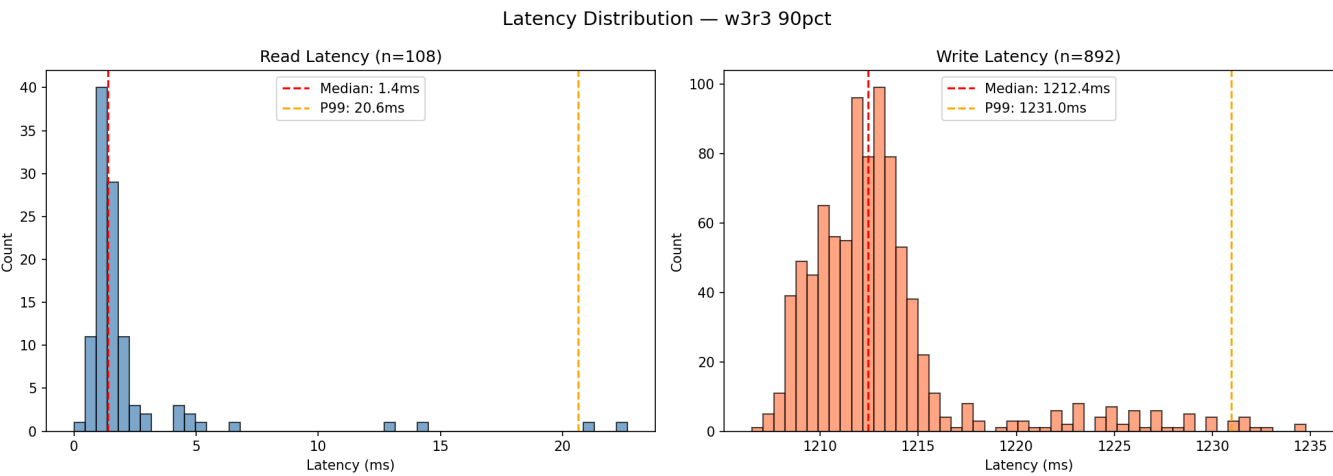


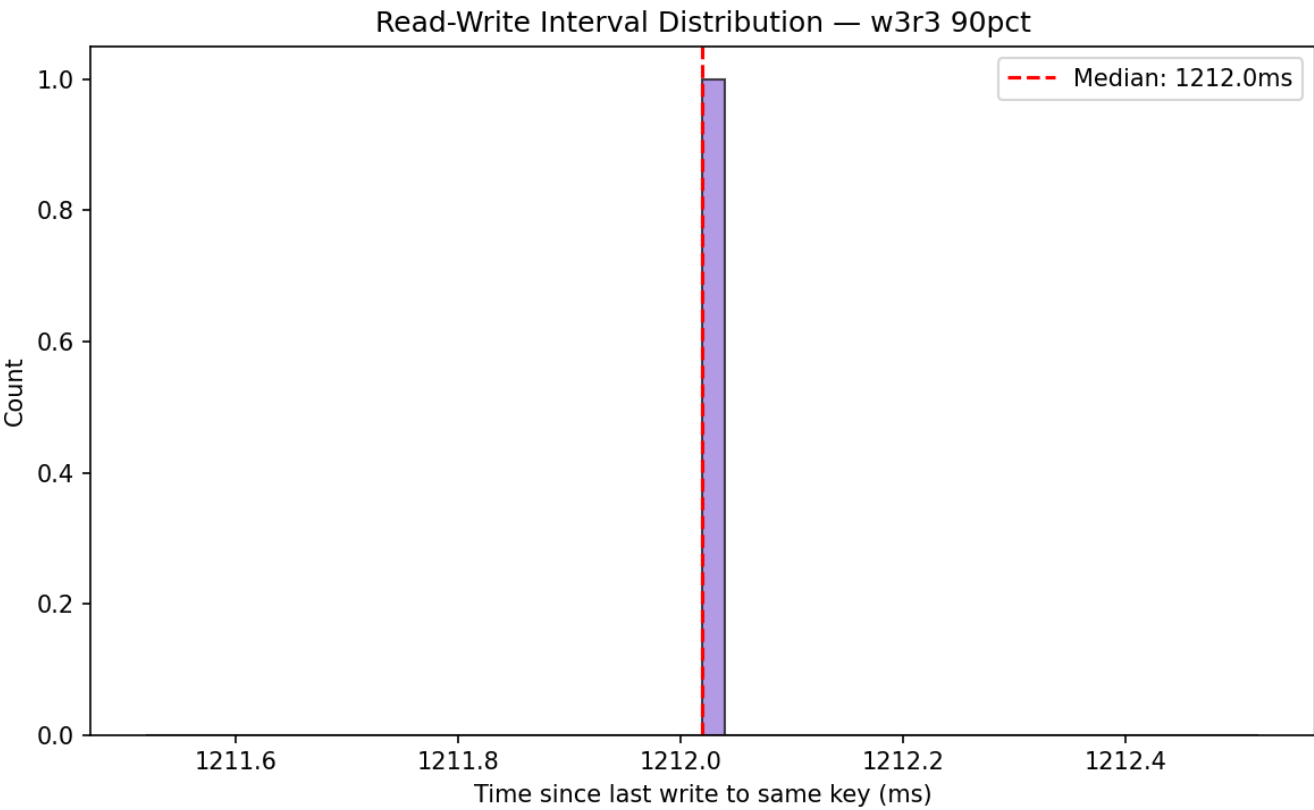
50% Writes / 50% Reads





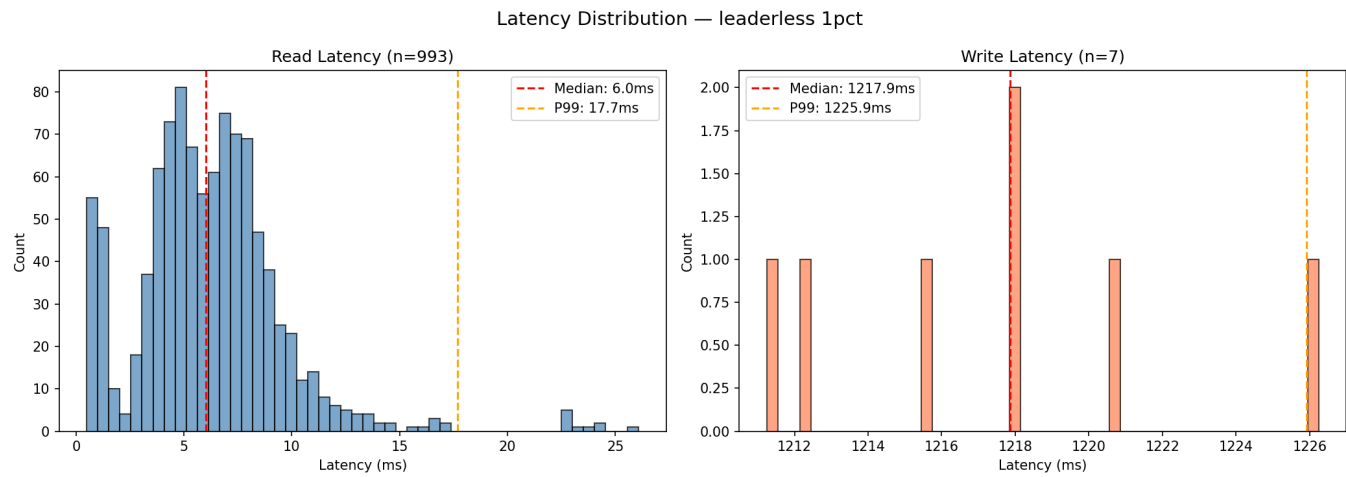
90% Writes / 10% Reads

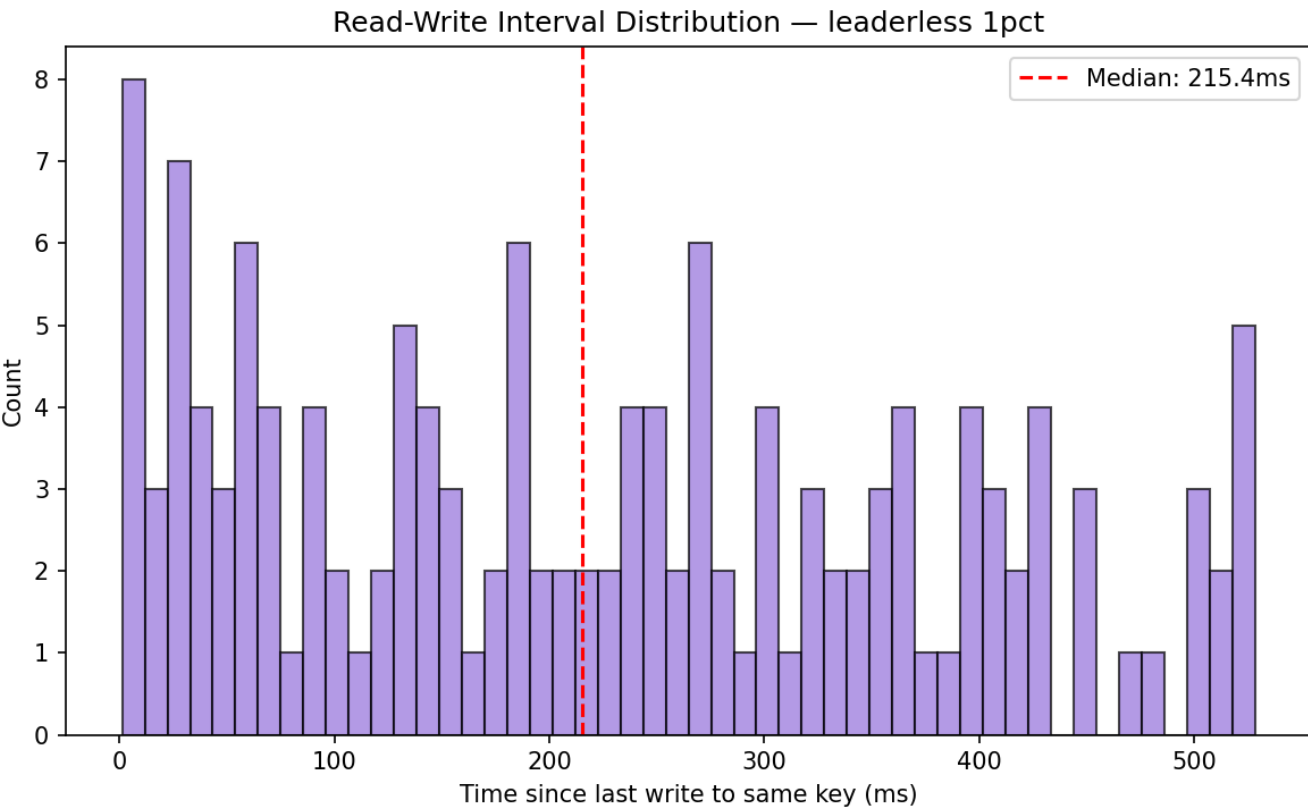




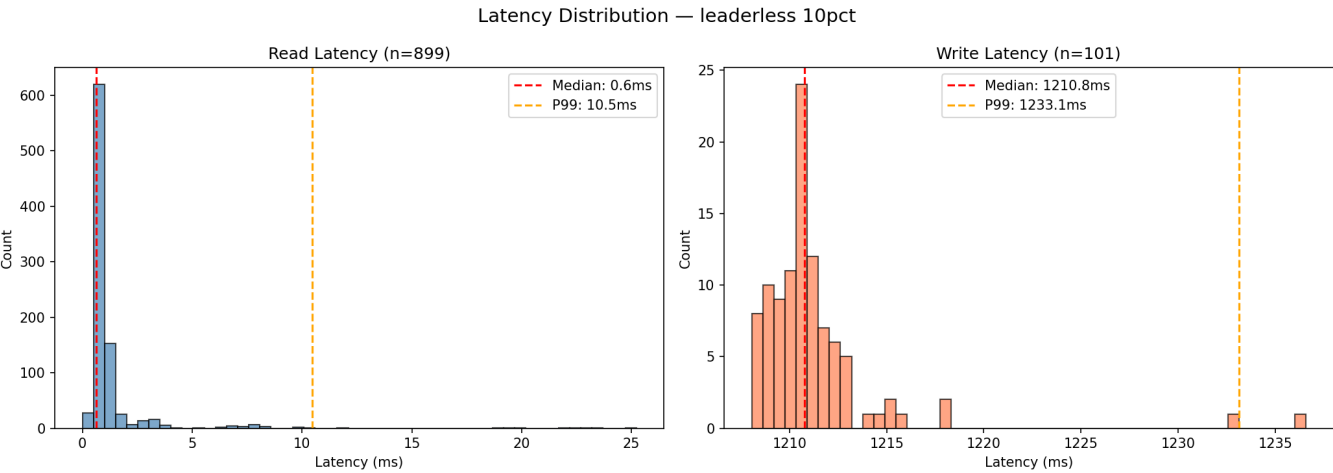
Graphs — Leaderless W=N, R=1

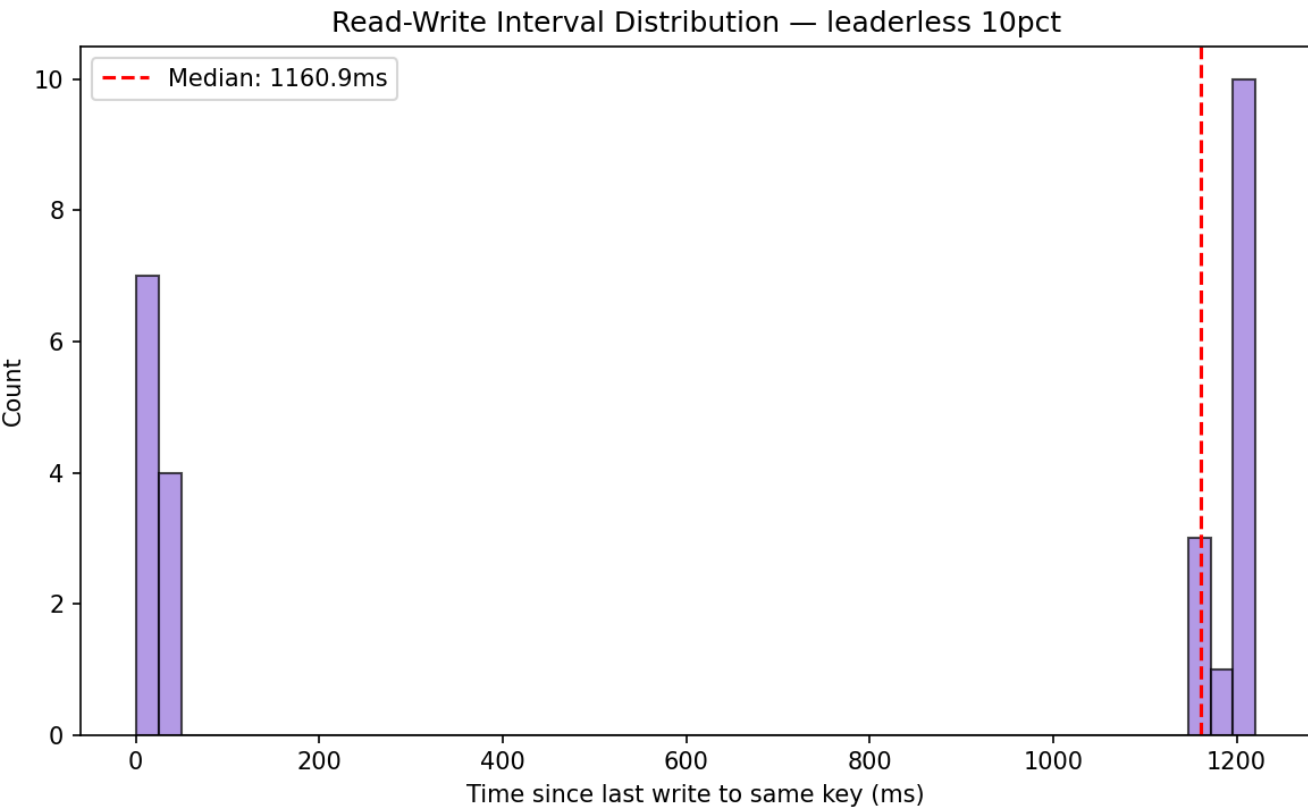
1% Writes / 99% Reads



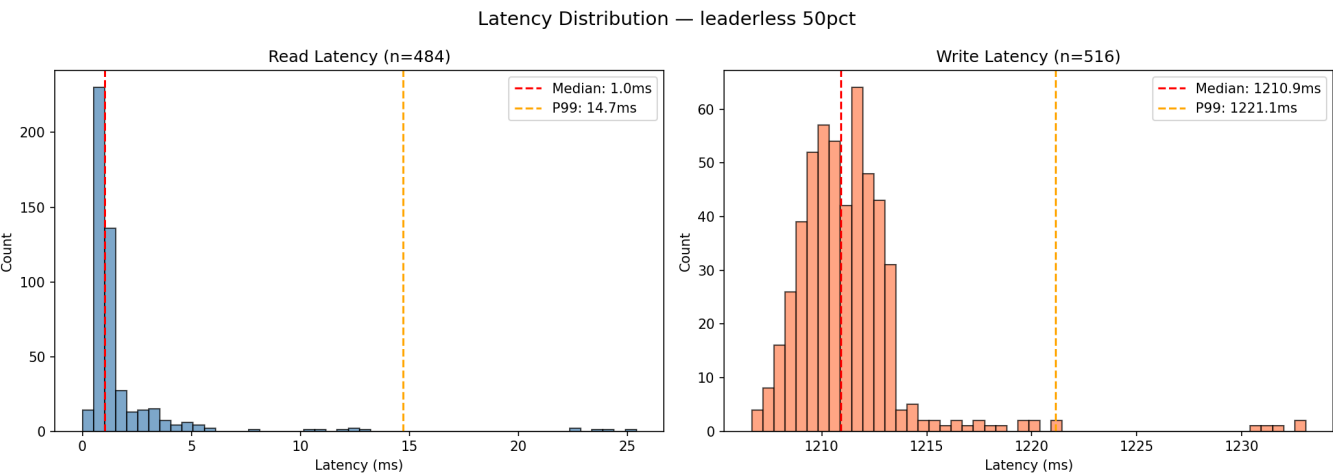


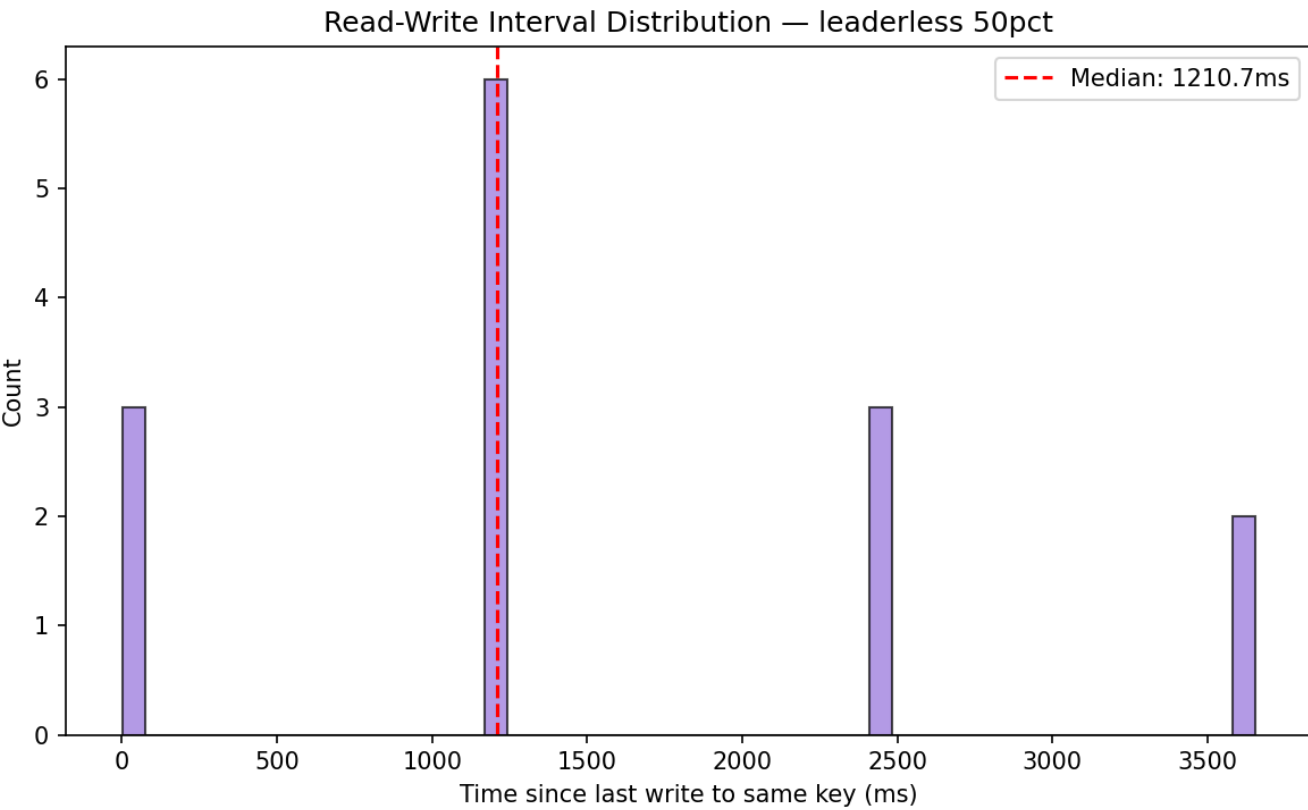
10% Writes / 90% Reads



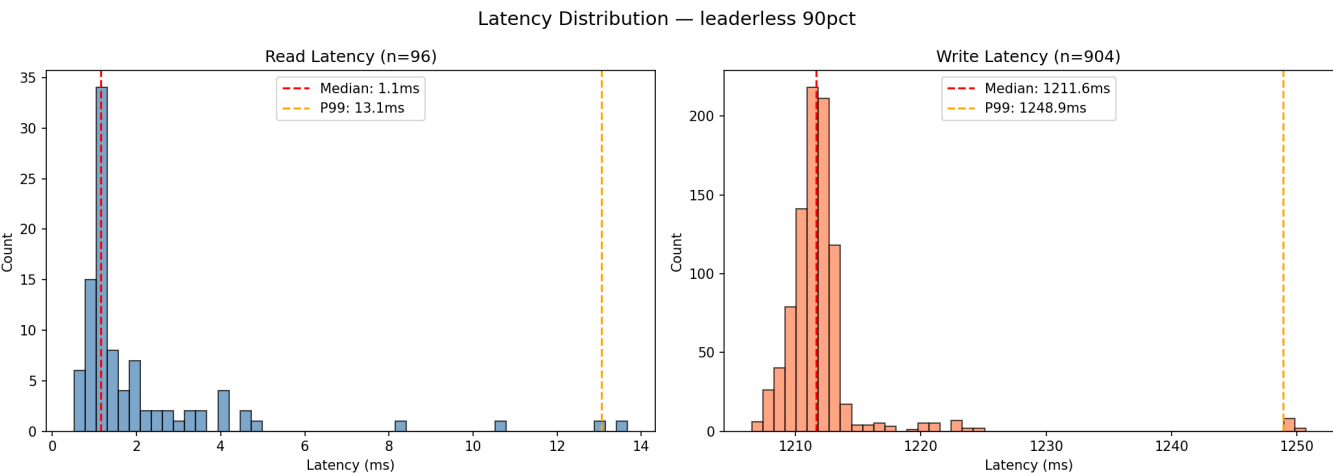


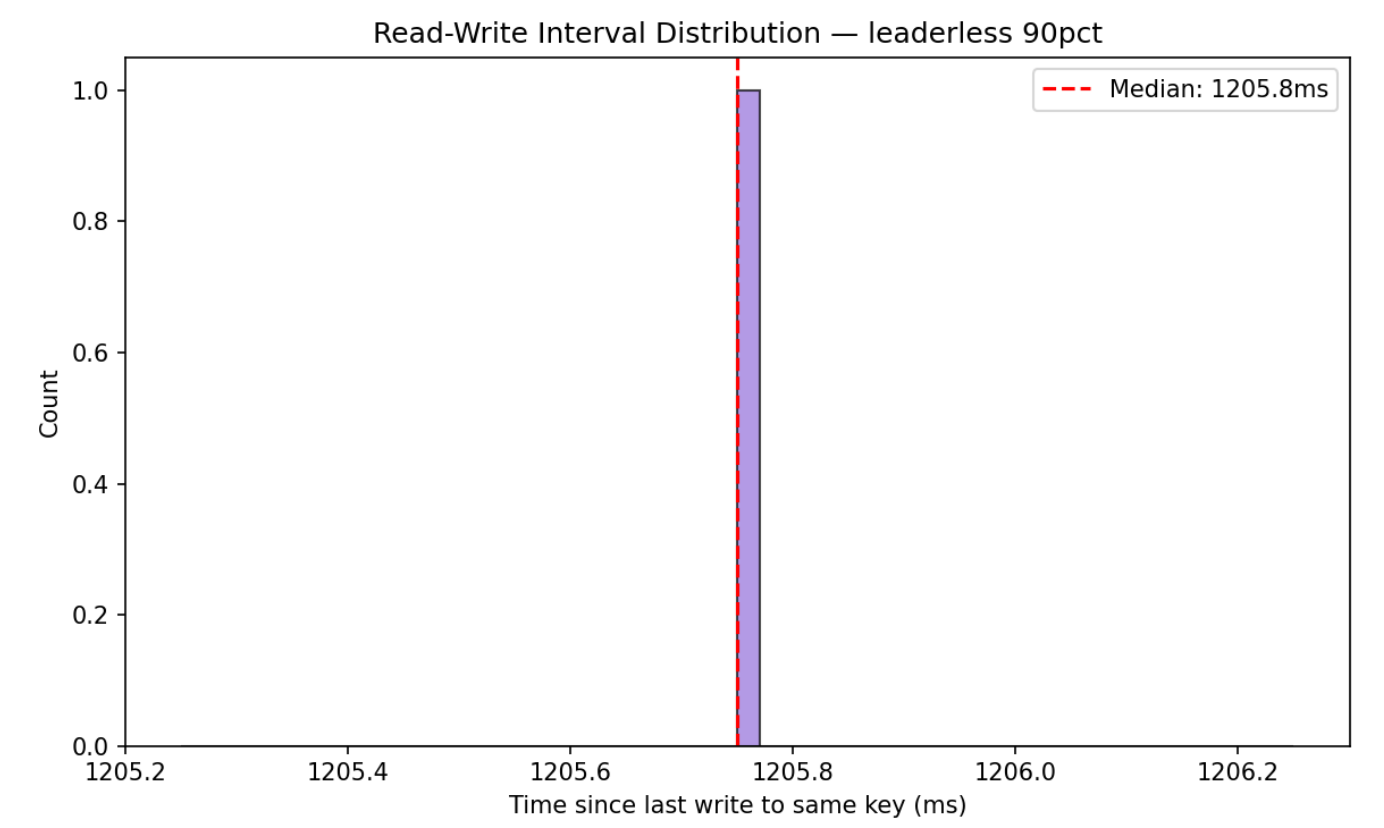
50% Writes / 50% Reads





90% Writes / 10% Reads





Screenshots

Load-Test Screenshot Contact Sheet

01_w5r1_1_writes

5Sr1 @ 1% writes

```
2026/04/06 01:02:30 Load Test Config: writes=1%, reads=99%, requests=1000, concurrency=10, keys=10
2026/04/06 01:02:30 Write nodes: [localhost:8080]
2026/04/06 01:02:30 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:02:33 ===== RESULTS =====
2026/04/06 01:02:33 Total time: 3.2083286s
2026/04/06 01:02:33 Requests: 1000 total (13 writes, 987 reads)
2026/04/06 01:02:33 Avg write latency: 1.215268784s
2026/04/06 01:02:33 Avg read latency: 8.642384ms
2026/04/06 01:02:33 Stale reads: 0
2026/04/06 01:02:33 Throughput: 311.7 req/s
2026/04/06 01:02:33 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Umf\results\results_w5r1_1pct.csv
```

02_w5r1_10_writes

5Sr1 @ 10% writes

```
2026/04/06 01:02:34 Load Test Config: writes=10%, reads=90%, requests=1000, concurrency=10, keys=10
2026/04/06 01:02:34 Write nodes: [localhost:8080]
2026/04/06 01:02:34 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:02:45 ===== RESULTS =====
2026/04/06 01:02:45 Total time: 11.0423345s
2026/04/06 01:02:45 Requests: 1000 total (86 writes, 914 reads)
2026/04/06 01:02:45 Avg write latency: 1.21611354s
2026/04/06 01:02:45 Avg read latency: 1.5627ms
2026/04/06 01:02:45 Stale reads: 6
2026/04/06 01:02:45 Throughput: 90.4 req/s
2026/04/06 01:02:45 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Umf\results\results_w5r1_10pct.csv
```

03_w5r1_50_writes

5Sr1 @ 50% writes

```
2026/04/06 01:02:45 Load Test Config: writes=50%, reads=50%, requests=1000, concurrency=10, keys=10
2026/04/06 01:02:45 Write nodes: [localhost:8080]
2026/04/06 01:02:45 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:03:47 ===== RESULTS =====
2026/04/06 01:03:47 Total time: 1m.8955093s
2026/04/06 01:03:47 Requests: 1000 total (500 writes, 491 reads)
2026/04/06 01:03:47 Avg write latency: 1.21197645s
2026/04/06 01:03:47 Avg read latency: 1.63697ms
2026/04/06 01:03:47 Stale reads: 75
2026/04/06 01:03:47 Throughput: 16.2 req/s
2026/04/06 01:03:47 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Umf\results\results_w5r1_50pct.csv
```

04_w5r1_90_writes

5Sr1 @ 90% writes

```
2026/04/06 01:03:48 Load Test Config: writes=90%, reads=10%, requests=1000, concurrency=10, keys=10
2026/04/06 01:03:48 Write nodes: [localhost:8080]
2026/04/06 01:03:48 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:05:37 ===== RESULTS =====
2026/04/06 01:05:37 Total time: 1m0.260514s
2026/04/06 01:05:37 Requests: 1000 total (900 writes, 100 reads)
2026/04/06 01:05:37 Avg write latency: 1.213686442s
2026/04/06 01:05:37 Avg read latency: 2.406343ms
2026/04/06 01:05:37 Stale reads: 26
2026/04/06 01:05:37 Throughput: 9.2 req/s
2026/04/06 01:05:37 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Umf\results\results_w5r1_90pct.csv
```

05_w1r5_1_writes

06_w1r5_10_writes

w1r5 @ 1% writes

```
2026/04/06 01:05:39 Load Test Config: writes=1%, reads=99%, requests=1000, concurrency=10, keys=10
2026/04/06 01:05:39 Write nodes: [localhost:8080]
2026/04/06 01:05:39 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:05:41 ===== RESULTS =====
2026/04/06 01:05:41 Total time: 2.5509066s
2026/04/06 01:05:41 Requests: 1000 total (111 writes, 987 reads)
2026/04/06 01:05:41 Avg write latency: 1.113781444s
2026/04/06 01:05:41 Avg read latency: 1.881248ms
2026/04/06 01:05:41 Stale reads: 0
2026/04/06 01:05:41 Throughput: 392.0 req/s
2026/04/06 01:05:41 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w1r5_1pct.csv
```

w1r5 @ 10% writes

```
2026/04/06 01:05:42 Load Test Config: writes=10%, reads=90%, requests=1000, concurrency=10, keys=10
2026/04/06 01:05:42 Write nodes: [localhost:8080]
2026/04/06 01:05:42 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:05:54 ===== RESULTS =====
2026/04/06 01:05:54 Total time: 12.2590277s
2026/04/06 01:05:54 Requests: 1000 total (99 writes, 901 reads)
2026/04/06 01:05:54 Avg write latency: 1.111342844s
2026/04/06 01:05:54 Avg read latency: 1.353657ms
2026/04/06 01:05:54 Stale reads: 31
2026/04/06 01:05:54 Throughput: 81.6 req/s
2026/04/06 01:05:54 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w1r5_10pct.csv
```

07_w1r5_50_writes

w1r5 @ 50% writes

```
2026/04/06 01:06:54 Load Test Config: writes=50%, reads=50%, requests=1000, concurrency=10, keys=10
2026/04/06 01:06:54 Write nodes: [localhost:8080]
2026/04/06 01:06:54 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:06:55 ===== RESULTS =====
2026/04/06 01:06:55 Total time: 1m.7118356s
2026/04/06 01:06:55 Requests: 1000 total (500 writes, 500 reads)
2026/04/06 01:06:55 Avg write latency: 1.211390066s
2026/04/06 01:06:55 Avg read latency: 1.83574ms
2026/04/06 01:06:55 Stale reads: 68
2026/04/06 01:06:55 Throughput: 16.5 req/s
2026/04/06 01:06:55 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w1r5_50pct.csv
```

08_w1r5_90_writes

w1r5 @ 90% writes

```
2026/04/06 01:06:55 Load Test Config: writes=90%, reads=10%, requests=1000, concurrency=10, keys=10
2026/04/06 01:06:55 Write nodes: [localhost:8080]
2026/04/06 01:06:55 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:08:45 ===== RESULTS =====
2026/04/06 01:08:45 Total time: 1m49.2280281s
2026/04/06 01:08:45 Requests: 1000 total (898 writes, 102 reads)
2026/04/06 01:08:45 Avg write latency: 1.211390281s
2026/04/06 01:08:45 Avg read latency: 2.463257ms
2026/04/06 01:08:45 Stale reads: 17
2026/04/06 01:08:45 Throughput: 9.2 req/s
2026/04/06 01:08:45 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w1r5_90pct.csv
```

09_w3r3_1_writes

w3r3 @ 1% writes

```
2026/04/06 01:08:46 Load Test Config: writes=1%, reads=99%, requests=1000, concurrency=10, keys=10
2026/04/06 01:08:46 Write nodes: [localhost:8080]
2026/04/06 01:08:46 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:08:48 ===== RESULTS =====
2026/04/06 01:08:48 Total time: 1.4915724s
2026/04/06 01:08:48 Requests: 1000 total (8 writes, 992 reads)
2026/04/06 01:08:48 Avg write latency: 1.11079115s
2026/04/06 01:08:48 Avg read latency: 2.177119ms
2026/04/06 01:08:48 Stale reads: 1
2026/04/06 01:08:48 Throughput: 670.4 req/s
2026/04/06 01:08:48 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w3r3_1pct.csv
```

10_w3r3_10_writes

w3r3 @ 10% writes

```
2026/04/06 01:08:48 Load Test Config: writes=10%, reads=90%, requests=1000, concurrency=10, keys=10
2026/04/06 01:08:48 Write nodes: [localhost:8080]
2026/04/06 01:08:48 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:09:00 ===== RESULTS =====
2026/04/06 01:09:00 Total time: 12.2664446s
2026/04/06 01:09:00 Requests: 1000 total (98 writes, 902 reads)
2026/04/06 01:09:00 Avg write latency: 1.11835131s
2026/04/06 01:09:00 Avg read latency: 1.36292ms
2026/04/06 01:09:00 Stale reads: 3
2026/04/06 01:09:00 Throughput: 81.5 req/s
2026/04/06 01:09:00 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w3r3_10pct.csv
```

11_w3r3_50_writes

w3r3 @ 50% writes

```
2026/04/06 01:09:01 Load Test Config: writes=50%, reads=50%, requests=1000, concurrency=10, keys=10
2026/04/06 01:09:01 Write nodes: [localhost:8080]
2026/04/06 01:09:01 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:09:59 ===== RESULTS =====
2026/04/06 01:09:59 Total time: 1m.2687382s
2026/04/06 01:09:59 Requests: 1000 total (476 writes, 524 reads)
2026/04/06 01:09:59 Avg write latency: 1.21117587s
2026/04/06 01:09:59 Avg read latency: 1.66079ms
2026/04/06 01:09:59 Stale reads: 74
2026/04/06 01:09:59 Throughput: 17.2 req/s
2026/04/06 01:09:59 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w3r3_50pct.csv
```

12_w3r3_90_writes

w3r3 @ 90% writes

```
2026/04/06 01:10:00 Load Test Config: writes=90%, reads=10%, requests=1000, concurrency=10, keys=10
2026/04/06 01:10:00 Write nodes: [localhost:8080]
2026/04/06 01:10:00 Read nodes: [localhost:8080 localhost:8081 localhost:8082 localhost:8083 localhost:8084]
2026/04/06 01:11:49 ===== RESULTS =====
2026/04/06 01:11:49 Total time: 1m49.2051796s
2026/04/06 01:11:49 Requests: 1000 total (892 writes, 108 reads)
2026/04/06 01:11:49 Avg write latency: 1.2111555s
2026/04/06 01:11:49 Avg read latency: 2.599833ms
2026/04/06 01:11:49 Stale reads: 16
2026/04/06 01:11:49 Throughput: 9.2 req/s
2026/04/06 01:11:49 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_w3r3_90pct.csv
```

13_leaderless_1_writes

leaderless @ 1% writes

```
2026/04/06 01:11:49 Load Test Config: writes=1%, reads=99%, requests=1000, concurrency=10, keys=10
2026/04/06 01:11:49 Write nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:11:49 Read nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:11:51 ===== RESULTS =====
2026/04/06 01:11:51 Total time: 2.8201723s
2026/04/06 01:11:51 Requests: 1000 total (7 writes, 993 reads)
2026/04/06 01:11:51 Avg write latency: 1.11141814s
2026/04/06 01:11:51 Avg read latency: 6.172876ms
2026/04/06 01:11:51 Stale reads: 8
2026/04/06 01:11:51 Throughput: 695.0 req/s
2026/04/06 01:11:51 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_leaderless_1pct.csv
```

14_leaderless_10_writes

leaderless @ 10% writes

```
2026/04/06 01:11:52 Load Test Config: writes=10%, reads=90%, requests=1000, concurrency=10, keys=10
2026/04/06 01:11:52 Write nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:11:52 Read nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:12:05 ===== RESULTS =====
2026/04/06 01:12:05 Total time: 13.4080147s
2026/04/06 01:12:05 Requests: 1000 total (161 writes, 839 reads)
2026/04/06 01:12:05 Avg write latency: 1.111184s
2026/04/06 01:12:05 Avg read latency: 1.207146ms
2026/04/06 01:12:05 Stale reads: 278
2026/04/06 01:12:05 Throughput: 74.6 req/s
2026/04/06 01:12:05 Results written to C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Vm9\results\results_leaderless_10pct.csv
```

15_leaderless_50_writes

```
Leaderless @ 50k writes
2026/04/06 01:12:06 Load Test Config: writes=50k, reads=50k, requests=1000k, concurrency=10, keys=10
2026/04/06 01:12:06 Write nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:12:06 Read nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:13:09 ===== RESULTS =====
2026/04/06 01:13:09 Total time: 1m 40.89998s
2026/04/06 01:13:09 Requests: 1000 total (516 writes, 484 reads)
2026/04/06 01:13:09 Avg write latency: 1.21123343s
2026/04/06 01:13:09 Avg read latency: 1.597563ms
2026/04/06 01:13:09 Stale reads: 228
2026/04/06 01:13:09 Throughput: 15.9 req/s
2026/04/06 01:13:09 Results written to C:\Users\mith\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Hw9\results\results_leaderless_50kpt.csv
```

16_leaderless_90_writes

```
Leaderless @ 90k writes
2026/04/06 01:13:09 Load Test Config: writes=90k, reads=10k, requests=1000k, concurrency=10, keys=10
2026/04/06 01:13:09 Write nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:13:09 Read nodes: [localhost:9080 localhost:9081 localhost:9082 localhost:9083 localhost:9084]
2026/04/06 01:14:59 ===== RESULTS =====
2026/04/06 01:14:59 Total time: 1m58.333892s
2026/04/06 01:14:59 Requests: 1000 total (984 writes, 96 reads)
2026/04/06 01:14:59 Avg write latency: 1.21888265s
2026/04/06 01:14:59 Avg read latency: 2.832413ms
2026/04/06 01:14:59 Stale reads: 53
2026/04/06 01:14:59 Throughput: 9.1 req/s
2026/04/06 01:14:59 Results written to C:\Users\mith\OneDrive\Desktop\DISTRIBUTED_SYSTEMS\Hw9\results\results_leaderless_90kpt.csv
```

extra_leader_follower_compose_ps_txt

```
Leader-Follower Compose Ps
NAME          IMAGE          COMMAND          SERVICE    CREATED        STATUS        PORTS
if-follower1  leader-follower-follower1  "/node"          follower1  56 seconds ago  Up 54 seconds  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
if-follower2  leader-follower-follower2  "/node"          follower2  56 seconds ago  Up 54 seconds  0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
if-follower3  leader-follower-follower3  "/node"          follower3  56 seconds ago  Up 54 seconds  0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
if-follower4  leader-follower-follower4  "/node"          follower4  56 seconds ago  Up 54 seconds  0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
if-leader     leader-follower-leader      "/node"          leader     56 seconds ago  Up 54 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
```

extra_leaderless_compose_ps_txt

```
Leaderless Compose Ps
NAME          IMAGE          COMMAND          SERVICE    CREATED        STATUS        PORTS
1l-node1      leaderless-node1  "/node"          node1      56 seconds ago  Up 54 seconds  0.0.0.0:9080->8080/tcp, [::]:9080->8080/tcp
1l-node2      leaderless-node2  "/node"          node2      56 seconds ago  Up 54 seconds  0.0.0.0:9081->8080/tcp, [::]:9081->8080/tcp
1l-node3      leaderless-node3  "/node"          node3      56 seconds ago  Up 54 seconds  0.0.0.0:9082->8080/tcp, [::]:9082->8080/tcp
1l-node4      leaderless-node4  "/node"          node4      56 seconds ago  Up 54 seconds  0.0.0.0:9083->8080/tcp, [::]:9083->8080/tcp
1l-node5      leaderless-node5  "/node"          node5      56 seconds ago  Up 54 seconds  0.0.0.0:9084->8080/tcp, [::]:9084->8080/tcp
```

extra_health_checks_txt

```
Health Checks
Leader-Follower leader:
{
  "node_id": "leader",
  "role": "leader",
  "status": "ok"
}

Leaderless node1:
{
  "node_id": "node1",
  "role": "leaderless",
  "status": "ok"
}
```

extra_leader_follower_smoke_txt

```
Leader-Follower Smoke
SET:
{"key":"smoke_1f","value":"alpha","version":2,"node_id":"leader"}

LEADER GET:
{"key":"smoke_1f","value":"alpha","version":2,"node_id":"leader"}

FOLLOWER GET:
{"key":"smoke_1f","value":"alpha","version":2,"node_id":"leader"}
```

extra_leaderless_smoke_txt

```
Leaderless Smoke
SET:
{"key":"smoke_1l","value":"beta","version":22,"node_id":"node2"}

COORDINATOR GET:
{"key":"smoke_1l","value":"beta","version":22,"node_id":"node2"}

PEER GET:
{"key":"smoke_1l","value":"beta","version":22,"node_id":"node5"}
```

extra_go_test_output_txt

```
Go Test Output
2026/04/06 01:00:48 Starting FOLLOWER node follower1 on :8081 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADER node leader on :8080 (w=5, R=1, followers=[localhost:8081 localhost:8082 localhost:8083 localhost:8084])
2026/04/06 01:00:48 Starting FOLLOWER node follower2 on :8082 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower3 on :8083 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower4 on :8084 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADERLESS node node2 on :9082 (peers=[localhost:9080 localhost:9082 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node3 on :9081 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node4 on :9083 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node5 on :9084 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9083])
=== RUN TestLeaderReadFromLeaderConsistent
2026/04/06 01:00:49 Configuration updated: w=5, R=1
--- PASS: TestLeaderReadFromLeaderConsistent (1.33s)
=== RUN TestLeaderFollowerGetConsistentAfterAck
2026/04/06 01:00:50 Configuration updated: w=5, R=1
--- PASS: TestLeaderFollowerGetConsistentAfterAck (1.37s)
=== RUN TestLeaderLocalReadOnInconsistencyWindow
2026/04/06 01:00:51 Configuration updated: w=1, R=1
--- PASS: TestLeaderLocalReadOnInconsistencyWindow (0.18s)
=== RUN TestLeaderQuorumReadReturnsLatestValue
2026/04/06 01:00:51 Configuration updated: w=3, R=3
--- PASS: TestLeaderQuorumReadReturnsLatestValue (0.76s)
=== RUN TestLeaderQuorumReadReturnsAtLeastThreeNodes
2026/04/06 01:00:52 Configuration updated: w=3, R=3
--- PASS: TestLeaderQuorumReadReturnsAtLeastThreeNodes (0.71s)
=== RUN TestLeaderLessInconsistencyWindow
--- PASS: TestLeaderLessInconsistencyWindow (1.23s)
=== RUN TestLeaderLessCoordinatorConsistentAfterAck
--- PASS: TestLeaderLessCoordinatorConsistentAfterAck (1.23s)
=== RUN TestLeaderLessCoordinatorConsistentAfterAck
--- PASS: TestLeaderLessCoordinatorConsistentAfterAck (1.23s)
```

```
--- PASS: TestLeaderlessAllNodesConsistentAfterAck (1.28s)
PASS
ok   @hw9/test@12.475s
```

Health Checks

Health Checks

Leader-Follower leader:

```
{
  "node_id": "leader",
  "role": "leader",
  "status": "ok"
}
```

Leaderless node1:

```
{
  "node_id": "node1",
  "role": "leaderless",
  "status": "ok"
}
```

Smoke Tests

Leader Follower Smoke

SET:

```
{"key": "smoke_1f", "value": "alpha", "version": 2, "node_id": "leader"}
```

LEADER GET:

```
{"key": "smoke_1f", "value": "alpha", "version": 2, "node_id": "leader"}
```

FOLLOWER GET:

```
{"key": "smoke_1f", "value": "alpha", "version": 2, "node_id": "leader"}
```

Leaderless Smoke

SET:

```
{"key": "smoke_1l", "value": "beta", "version": 22, "node_id": "node2"}
```

COORDINATOR GET:

```
{"key": "smoke_1l", "value": "beta", "version": 22, "node_id": "node2"}
```

PEER GET:

```
{"key": "smoke_1l", "value": "beta", "version": 22, "node_id": "node5"}
```